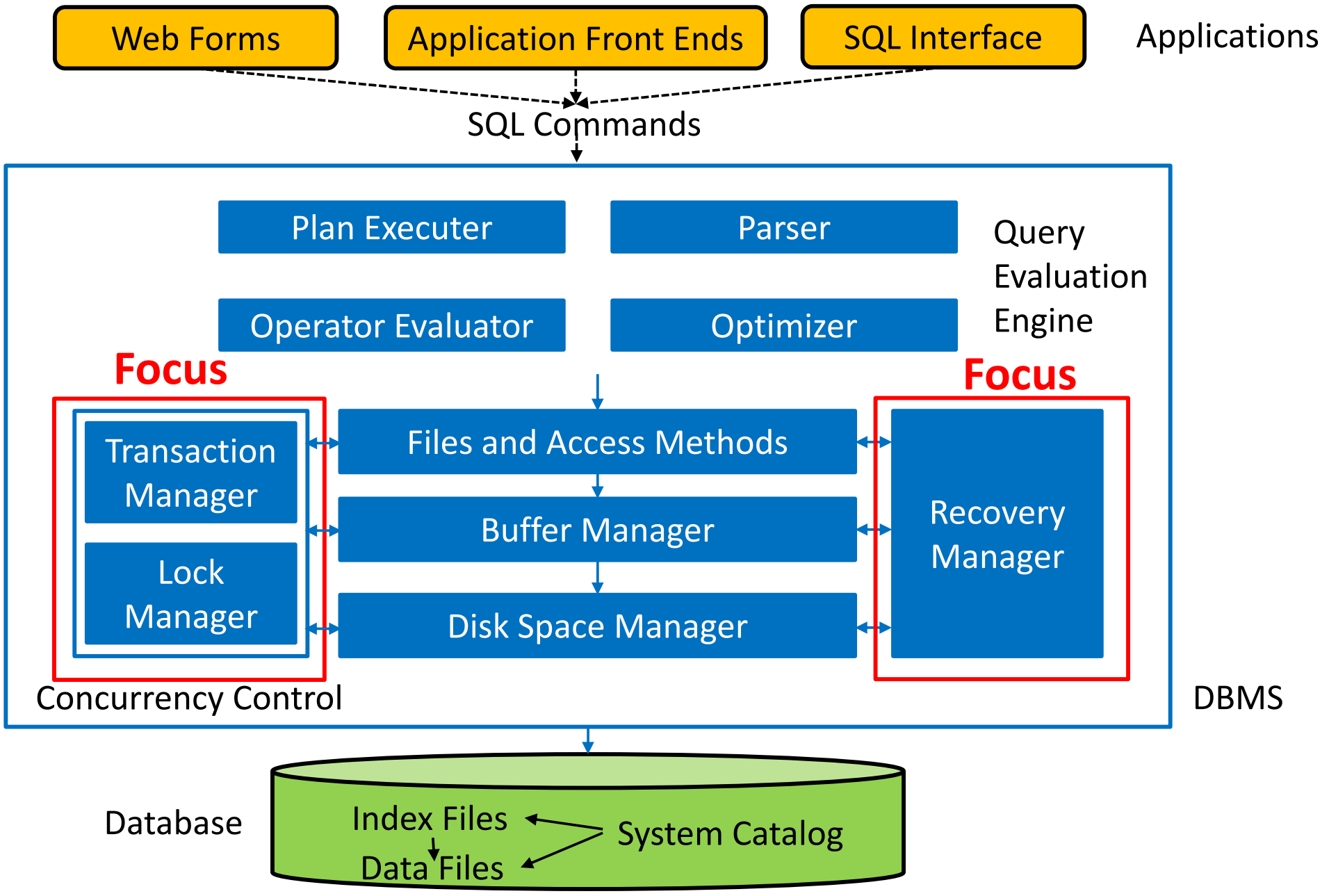
Lecture 7
Concurrency Control & Crash Recovery

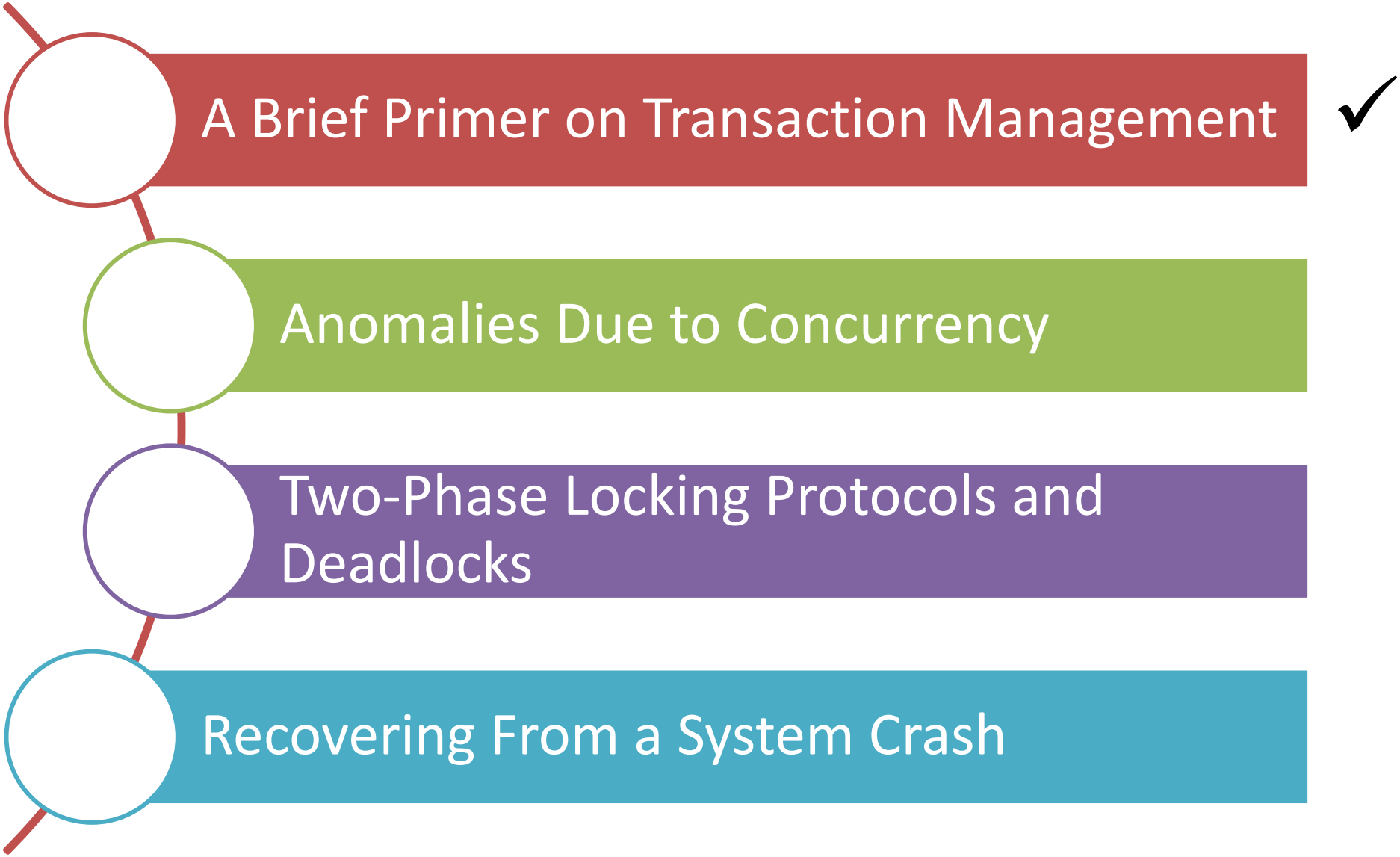
Dr. Yu Yang
Department of Mathematics and Information Technology
The Education University of Hong Kong

All teaching and learning materials of INT6164 MUST NOT be distributed or shared without the permission of the Instructor!

The Architecture of a Relational DBMS



Outline


A Brief Primer on Transaction Management
✓
Anomalies Due to Concurrency
Two-Phase Locking Protocols and Deadlocks
Recovering From a System Crash

Concurrent Execution of Programs

- A database is typically *shared* by a large number of users

- DBMSs *schedule* users' programs *concurrently*
  - While one user program is waiting for an I/O access to be satisfied, the CPU can process another program
    - Better system throughput

  - Interleaved executions of short and long programs allow the short program to complete quickly
    - Better response time
    - Fairer

Transactions

- *Any <u>one</u> execution* of a user program in a DBMS is denoted as a transaction
  - Executing the same program several times will generate several transactions

- A transaction is the basic unit of change as seen by a DBMS
  - E.g., Transfer $100 from account A to account B

- A transaction may carry out many operations on data, but DBMSs are only concerned about *reads* and *writes*

- Thus, in essence a transaction becomes *a sequence of reads and writes*

Transactions (*Cont'd*)

- In addition to reading and writing, a transaction must specify as its final action:
  - Either *Commit* (i.e., complete successfully)
  - Or *Abort* (i.e., terminate and *undo* actions)

- We make two assumptions:
  - Transactions interact only via database reads and writes (i.e., no *message passing*)
  - A database is a fixed collection of *independent* objects (A, B, C, etc.)

Schedules 调度

- A schedule is a list of actions (i.e., read, write, abort, and/or commit) from a *set* of transactions

- The *order* in which two actions of a transaction ***T*** appear in a schedule must be the same as they appear in ***T*** itself

- Assume **T1** = [R(A), W(A)] and **T2** = [R(B), W(B), R(C), W(C)]

T1	T2
R(A)	
W(A)	
	R(B)
	W(B)
	R(C)
	W(C)



T1	T2
R(A)	
W(A)	
	R(B)
	W(B)
	R(C)
	W(C)

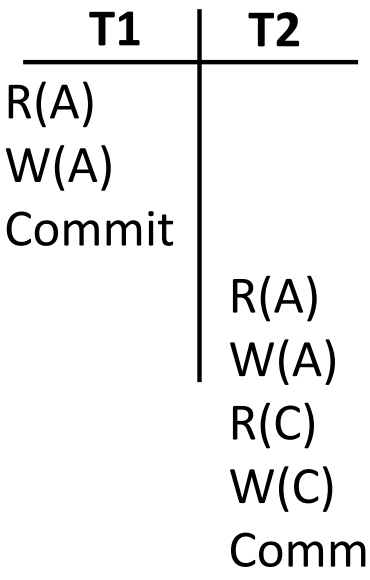


T1	T2
R(A)	
W(A)	
	R(C)
	W(C)
	R(B)
	W(B)

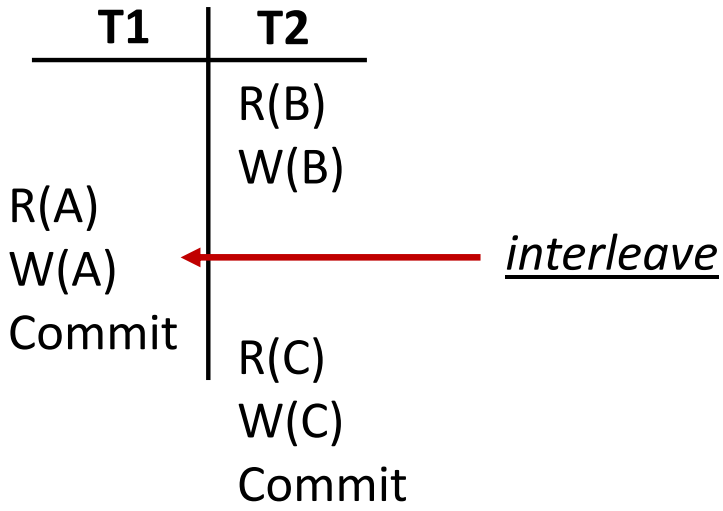


Serial Schedules 串行调度

- A complete schedule must contain all the actions of every transaction that appears on it
- If the actions of different transactions are *not interleaved*, the schedule is called a serial schedule



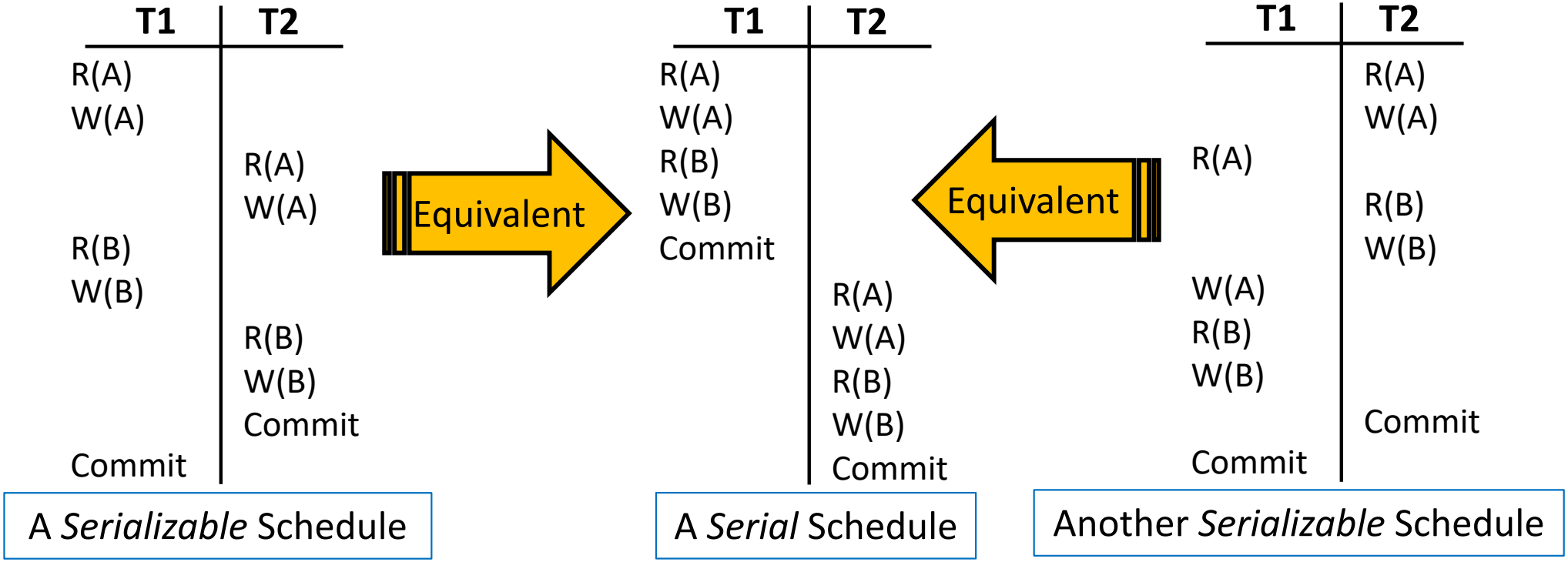
A Serial Schedule



A Non-Serial Schedule

Serializable Schedules

- Two schedules are said to be *equivalent* if for any database state, the effect of executing the 1st schedule is identical to the effect of executing the 2nd schedule
- A serializable schedule is a schedule that is equivalent to a serial schedule



Examples

- Assume transactions T1 and T2 as follows:

```
T1:    BEGIN   A=A-100,   B=B +100   END
T2:    BEGIN   A=1.06*A,  B=1.06*B   END
```

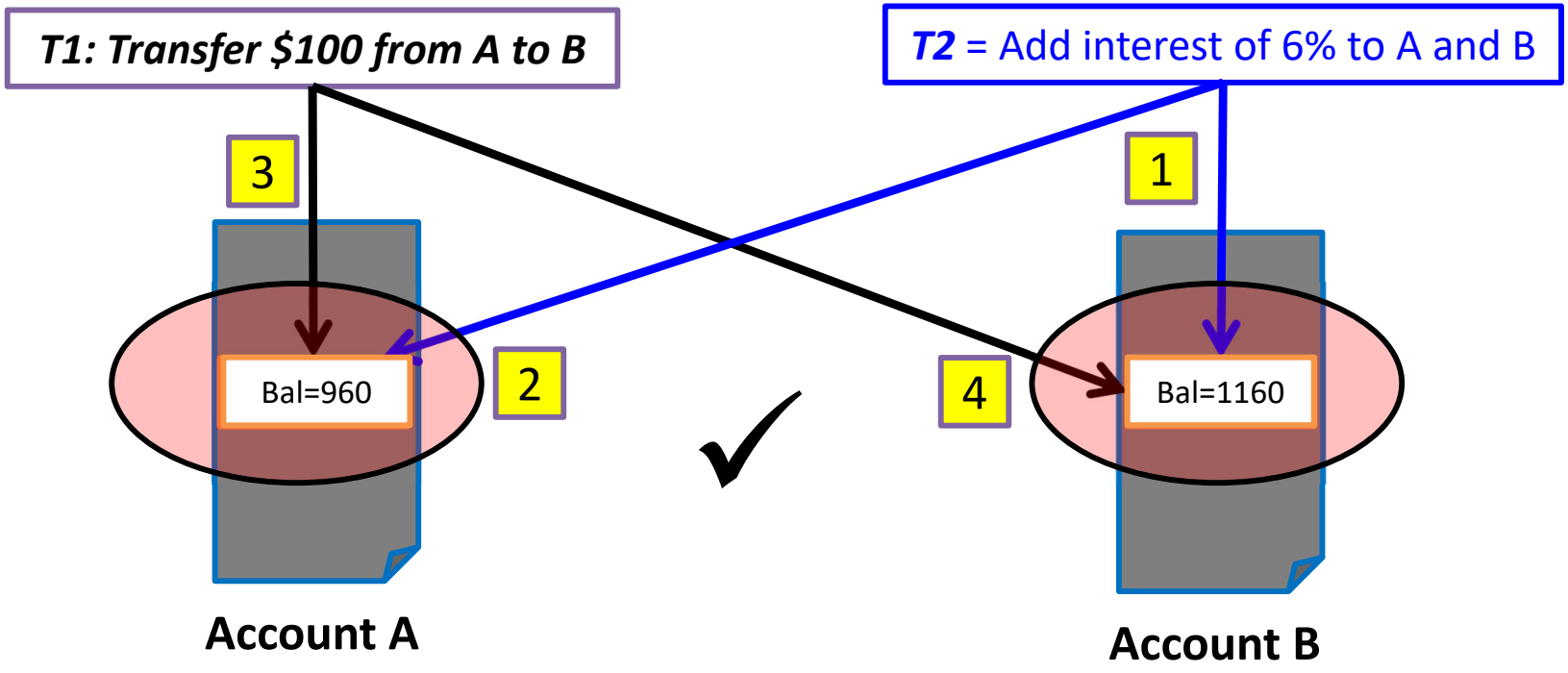
- T1 can be thought of as transferring $100 from A's account to B's account

- T2 can be thought of as crediting accounts A and B with a 6% interest payment

Examples: A *Serial* Schedule

- Assume transactions T1 and T2 as follows:

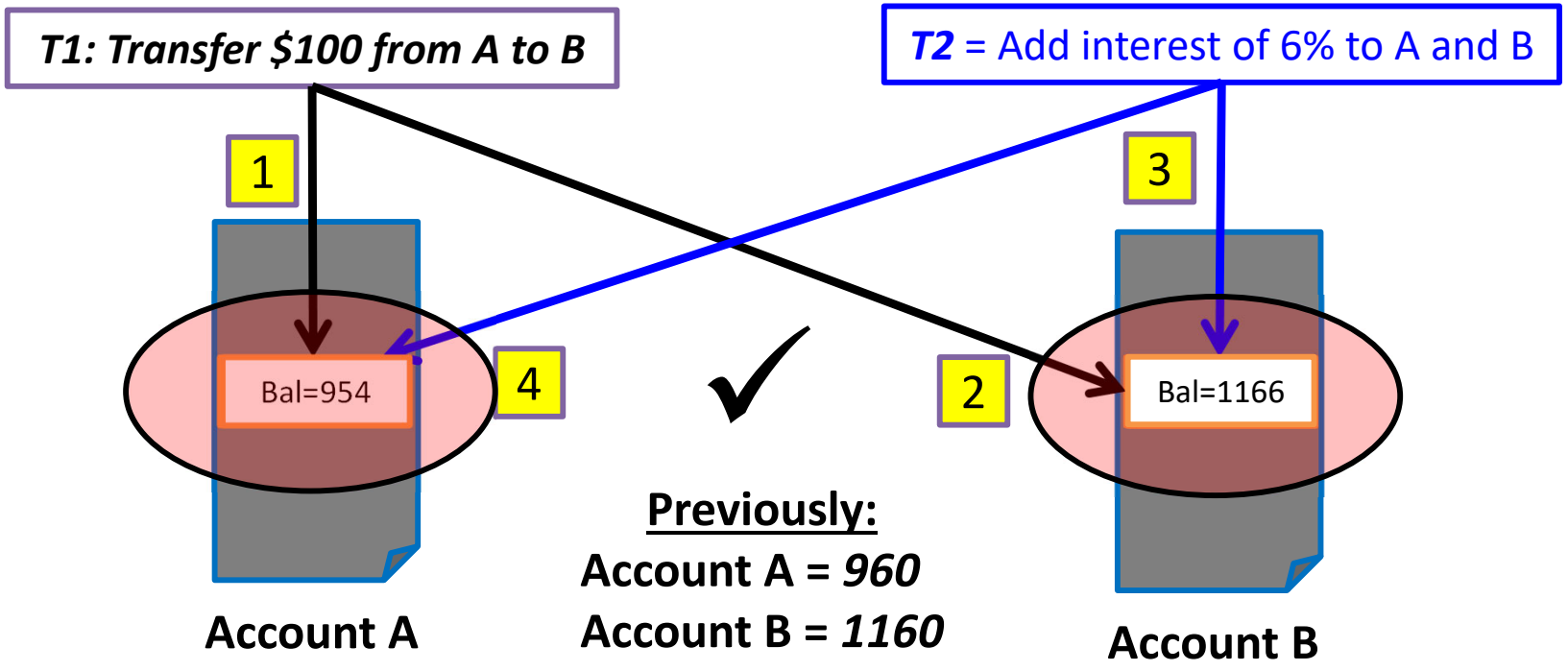
```
T1:    BEGIN  A=A-100,   B=B +100   END
T2:    BEGIN  A=1.06*A,  B=1.06*B  END
```



Examples: Another *Serial* Schedule

- Assume transactions T1 and T2 as follows:

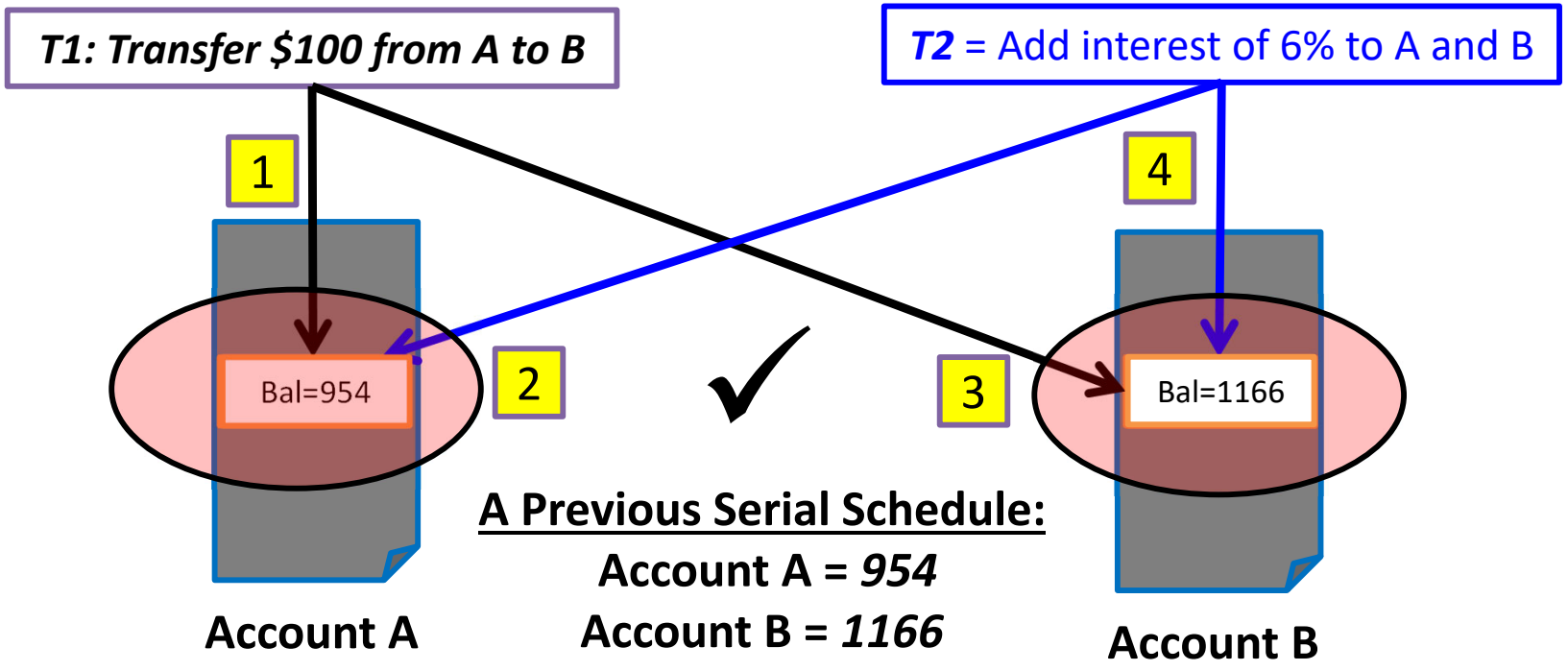
T1:	BEGIN	A=A-100,	B=B +100	END
T2:	BEGIN	A=1.06*A,	B=1.06*B	END



Examples: A *Serializable* Schedule

- Assume transactions T1 and T2 as follows:

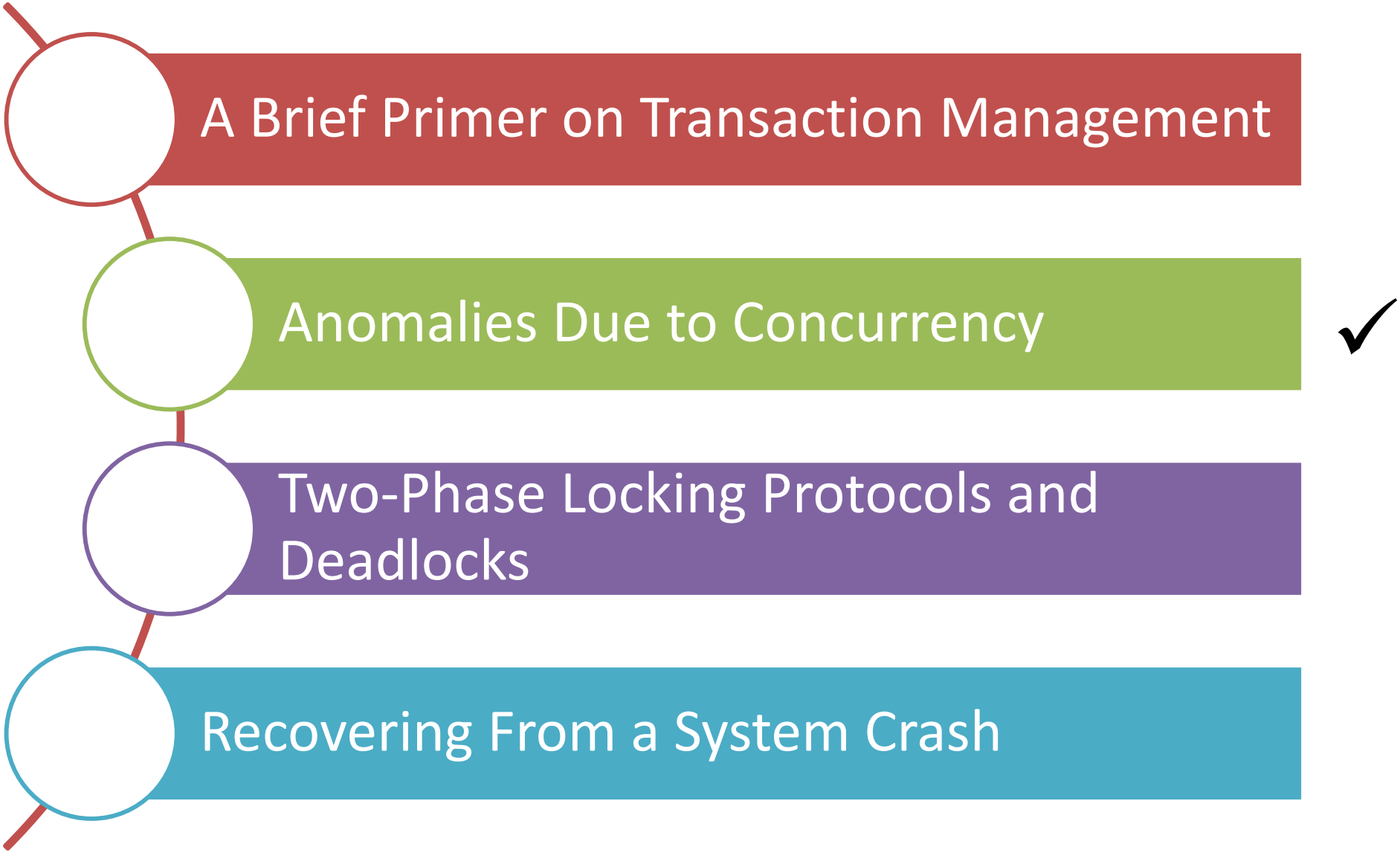
T1:	BEGIN	A=A-100,	B=B +100	END
T2:	BEGIN	A=1.06*A,	B=1.06*B	END



Comments

- There is no guarantee that T1 will execute before T2 or vice-versa, if both are submitted together
- However, the net effect *must* be equivalent to these two transactions running *serially* in some order
- Executing transactions serially in different orders may produce different results, but they are all acceptable!
- The DBMS makes no guarantees about which result will be the outcome of an interleaved execution

Outline


A Brief Primer on Transaction Management
Anomalies Due to Concurrency
Two-Phase Locking Protocols and Deadlocks
Recovering From a System Crash

✓

Anomalies 异常

- Interleaving actions of different transactions can leave the database in an inconsistent state

- Two actions on the same data object are said to ***conflict*** if at least one of them is a write

- There are 3 anomalies that can arise upon interleaving actions of different transactions (say, T1 and T2):
  - Write-Read (WR) Conflict: T2 reads a data object previously written by T1
  - Read-Write (RW) Conflict: T2 writes a data object previously read by T1
  - Write-Write (WW) Conflict: T2 writes a data object previously written by T1

Reading Uncommitted Data: WR Conflicts

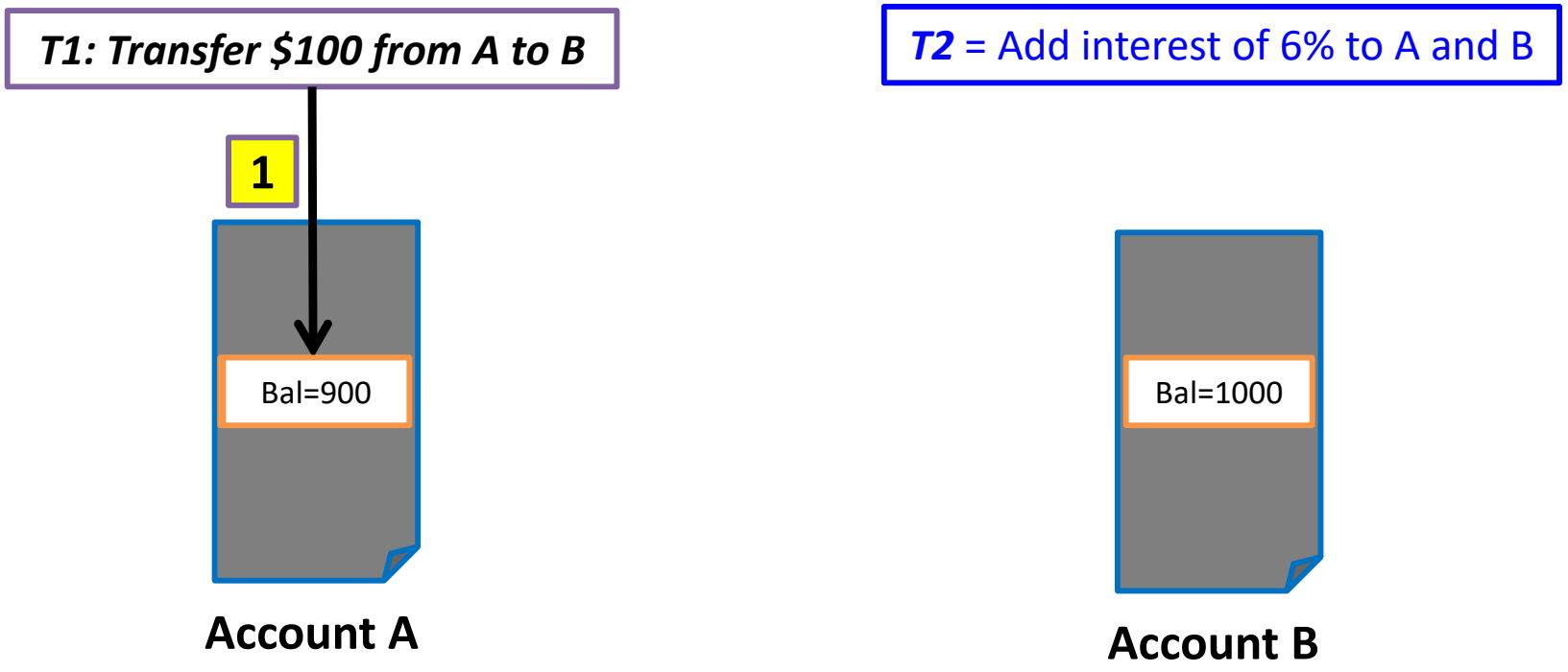
- WR conflicts arise when transaction T2 reads a data object A that has been modified by another transaction T1, *which has not yet committed*
  - Such a read is called a dirty read 脏读

- Assume T1 and T2 such that:
  - T1 transfers $100 from A's account to B's account
  - T2 credits accounts A and B with a 6% interest payment

```
T1:    BEGIN   A=A-100,   B=B +100   END
T2:    BEGIN   A=1.06*A,   B=1.06*B   END
```

Reading Uncommitted Data: WR Conflicts

- Suppose that T1 and T2 actions are *interleaved* as follows:
  - T1 deducts $100 from account A
  - T2 adds 6% interest to accounts A and B
  - T1 credits $100 to account B

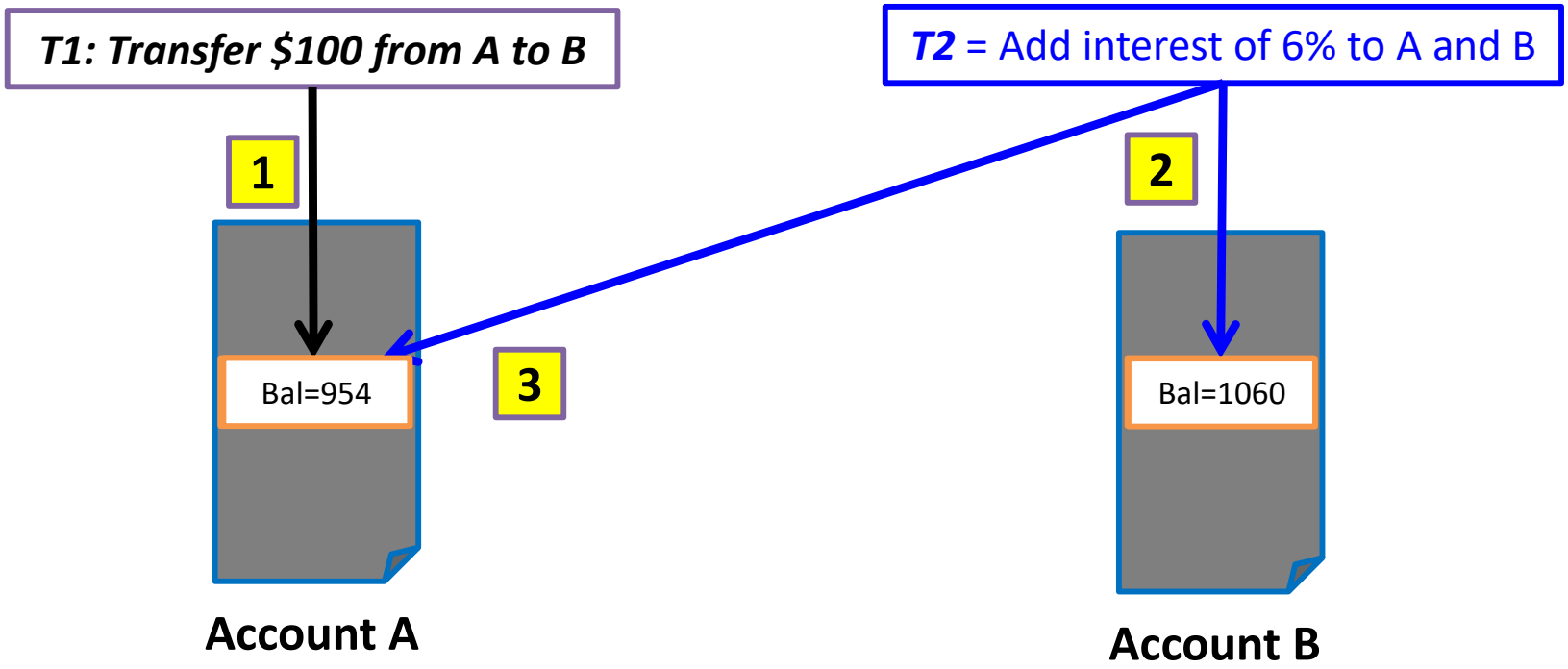


Reading Uncommitted Data: WR Conflicts

- Suppose that T1 and T2 actions are *interleaved* as follows:
  - T1 deducts $100 from account A
  - T2 adds 6% interest to accounts A and B
  - T1 credits $100 to account B

1

2 and 3



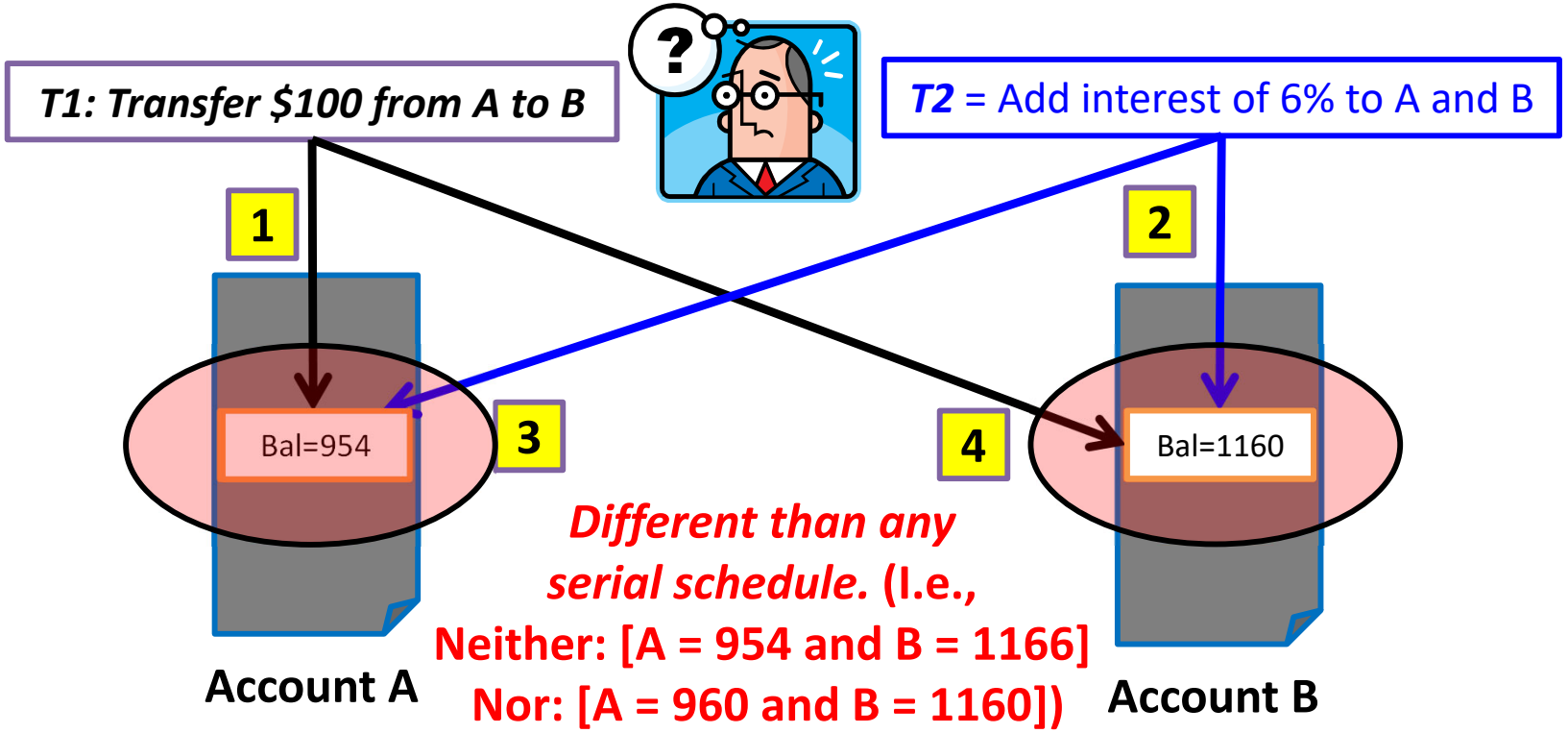
Reading Uncommitted Data: WR Conflicts

- Suppose that T1 and T2 actions are *interleaved* as follows:
  - T1 deducts $100 from account A
  - T2 adds 6% interest to accounts A and B
  - T1 credits $100 to account B

1

2 and 3

4



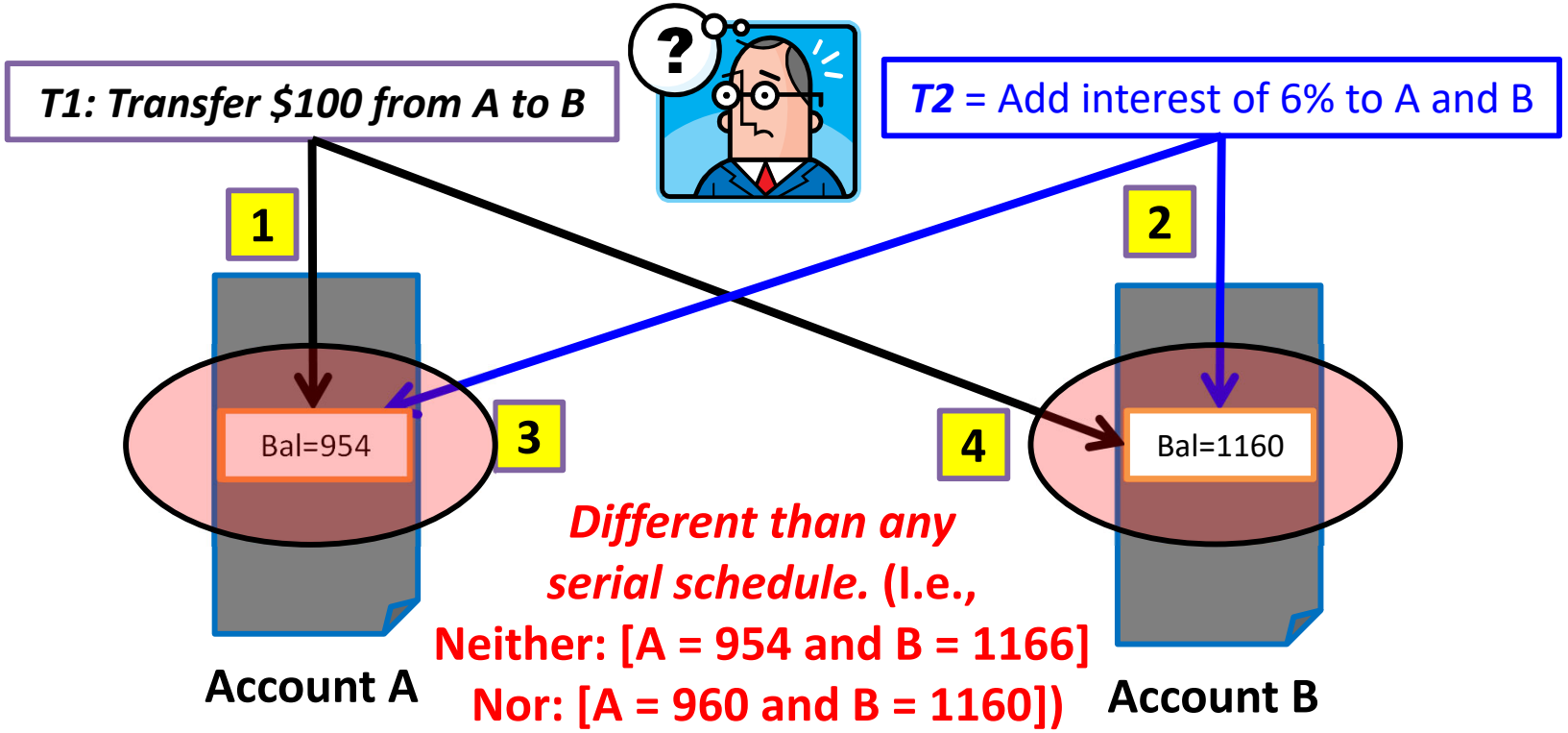
Reading Uncommitted Data: WR Conflicts

- Suppose that T1 and T2 actions are *interleaved* as follows:
  - T1 deducts $100 from account A
  - T2 adds 6% interest to accounts A and B
  - T1 credits $100 to account B

1

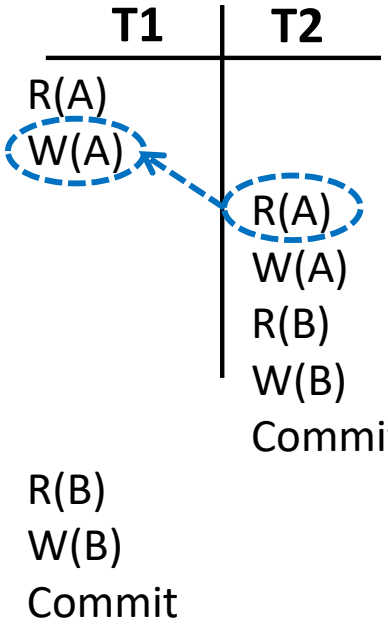
2 and 3

4



Reading Uncommitted Data: WR Conflicts

- T1 and T2 can be represented by the following schedule:



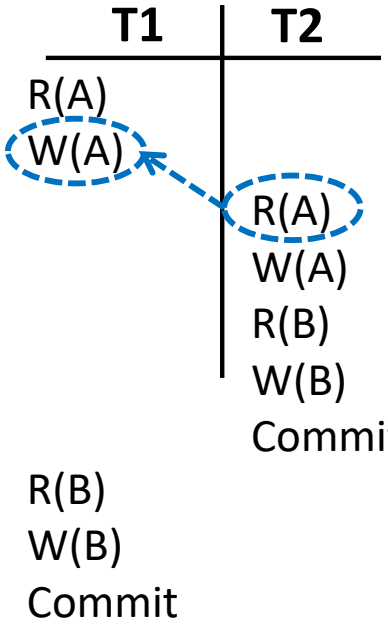
The value of A written by T1 is read by T2 before T1 has completed all its changes!

Why is this a problem?

- T1 may write some value into A that makes the database inconsistent
- As long as T1 overwrites this value with a 'correct' value of A before committing, no harm is done if T1 and T2 are run in some serial order (this is because T2 would then not see the temporary inconsistency)

Reading Uncommitted Data: WR Conflicts

- T1 and T2 can be represented by the following schedule:



The value of A written by T1 is read by T2 before T1 has completed all its changes!

Why is this a problem?

Note that although a transaction must leave a database in a consistent state *after* it completes, it is not required to keep the database consistent while it is still in progress!

Unrepeatable Reads: RW Conflicts

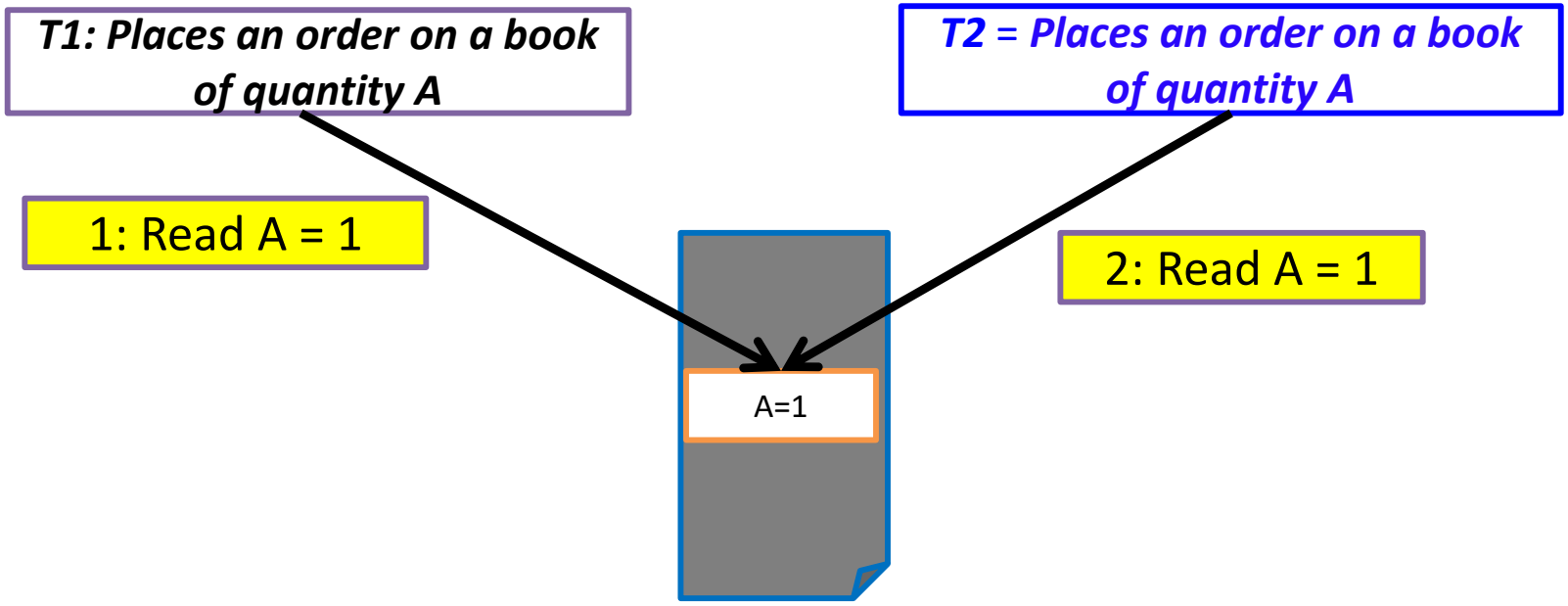
- RW conflicts arise when transaction T2 writes a data object A that has been read by another transaction T1, *while T1 is still in progress*

- If T1 tries to read A again, it will get a different result!
  - Such a read is called an unrepeatable read 不可重复读

- Assume A is the number of available copies for a book
  - A transaction that places an order on the book reads A, checks that A > 0 and decrements A
  - Assume two transactions, T1 and T2

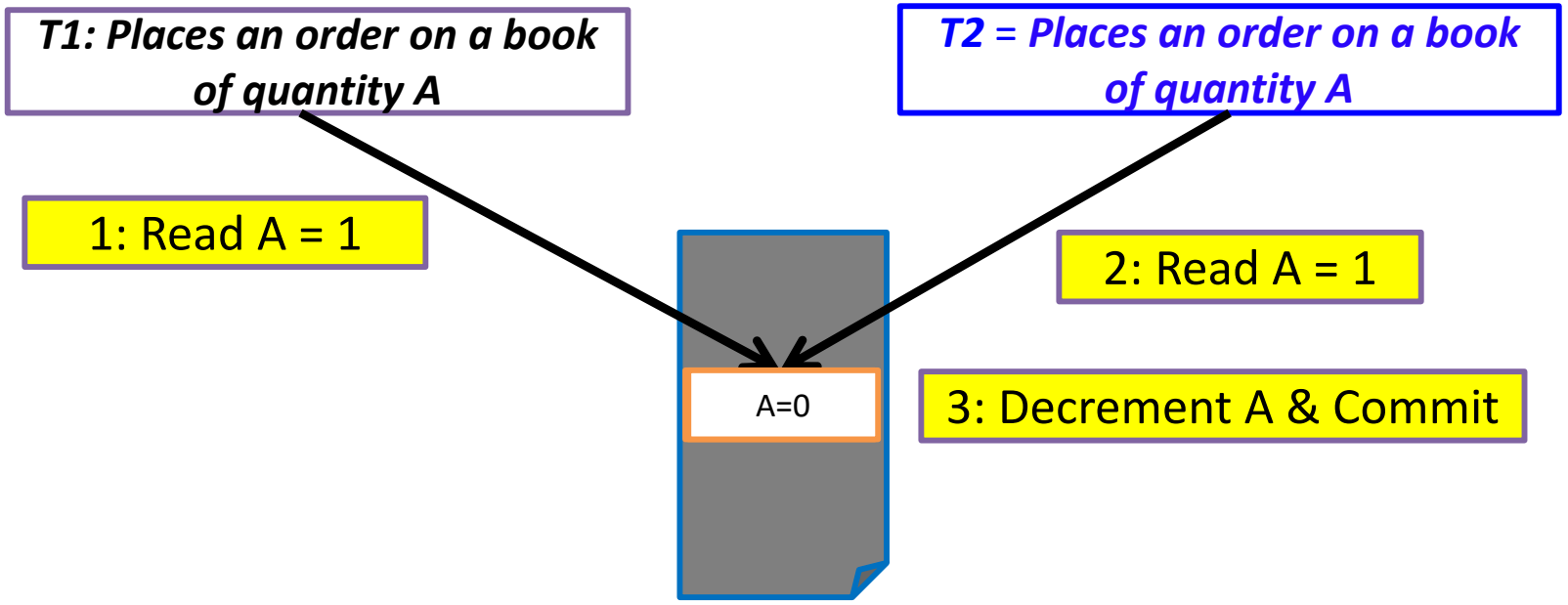
Unrepeatable Reads: RW Conflicts

- Suppose that T1 and T2 actions are interleaved as follows:
  - T1 reads A
  - T2 reads A, decrements A and commit
  - T1 tries to decrement A



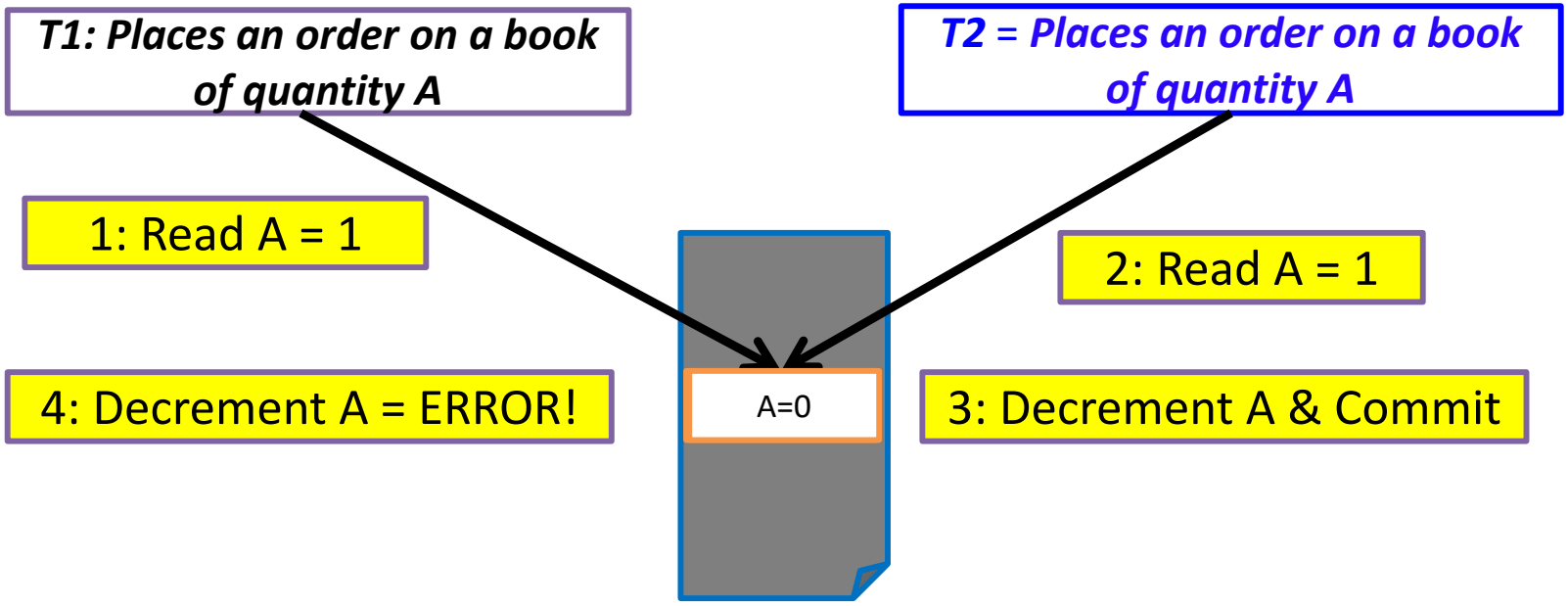
Unrepeatable Reads: RW Conflicts

- Suppose that T1 and T2 actions are interleaved as follows:
  - T1 reads A
  - T2 reads A, decrements A and commit
  - T1 tries to decrement A



Unrepeatable Reads: RW Conflicts

- Suppose that T1 and T2 actions are interleaved as follows:
  - T1 reads A
  - T2 reads A, decrements A and commit
  - T1 tries to decrement A



This situation will never arise in a serial execution of T1 and T2;
T2 would read A and see 0 and therefore not proceed with placing an order!

Overwriting Uncommitted Data: WW Conflicts

- WW conflicts arise when transaction T2 writes a data object A that has been written by another transaction T1, *while T1 is still in progress*

- Suppose that Tom and Jerry are two employees and their salaries *must be kept equal*

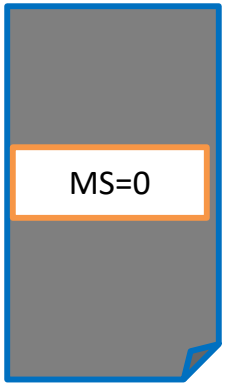
- Assume T1 sets Tom's and Jerry's salaries to $1000

- Assume T2 sets Tom's and Jerry's salaries to $2000

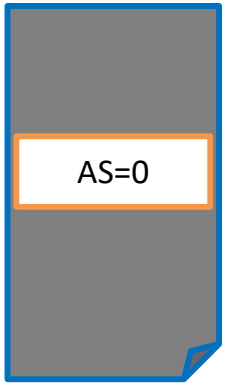
Overwriting Uncommitted Data: WW Conflicts

T1: Sets Salaries to $1000

T2 = Sets Salaries to $2000

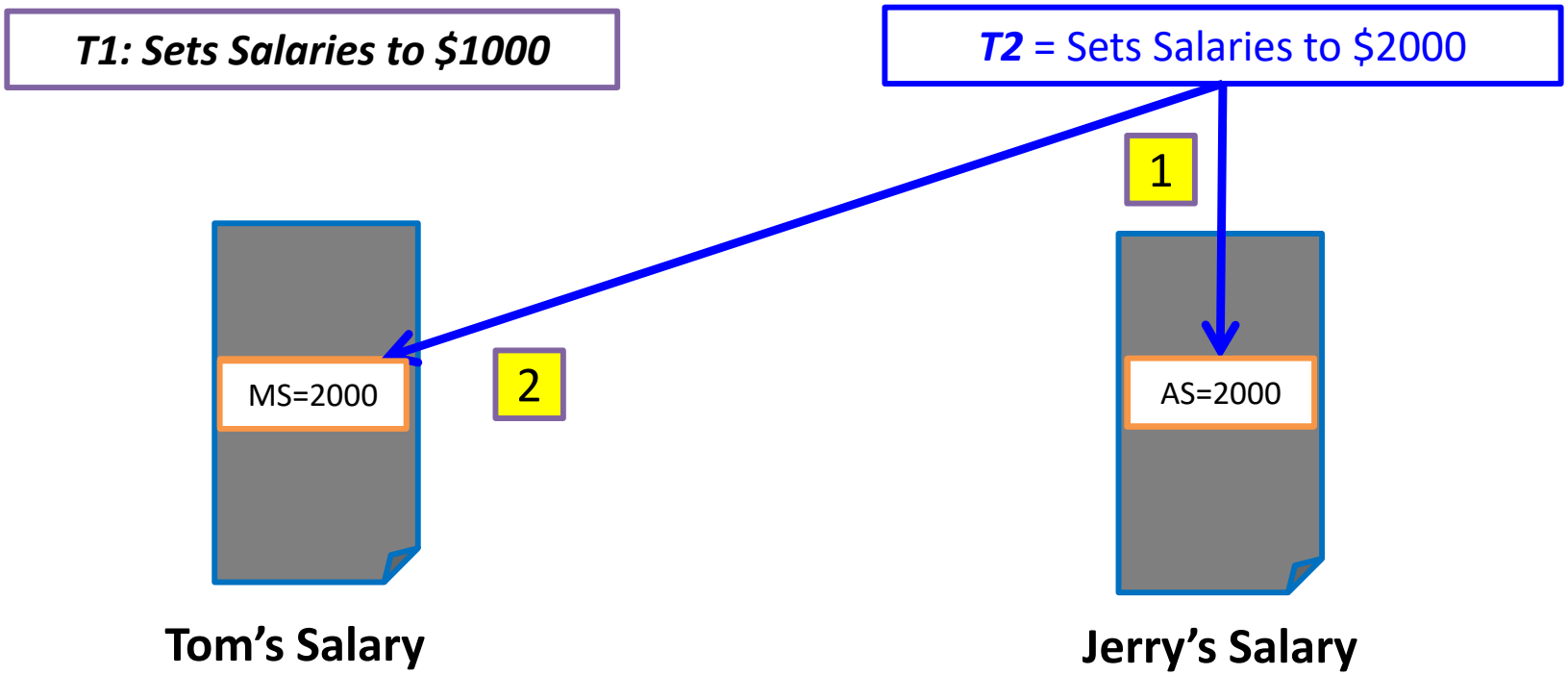


Tom's Salary

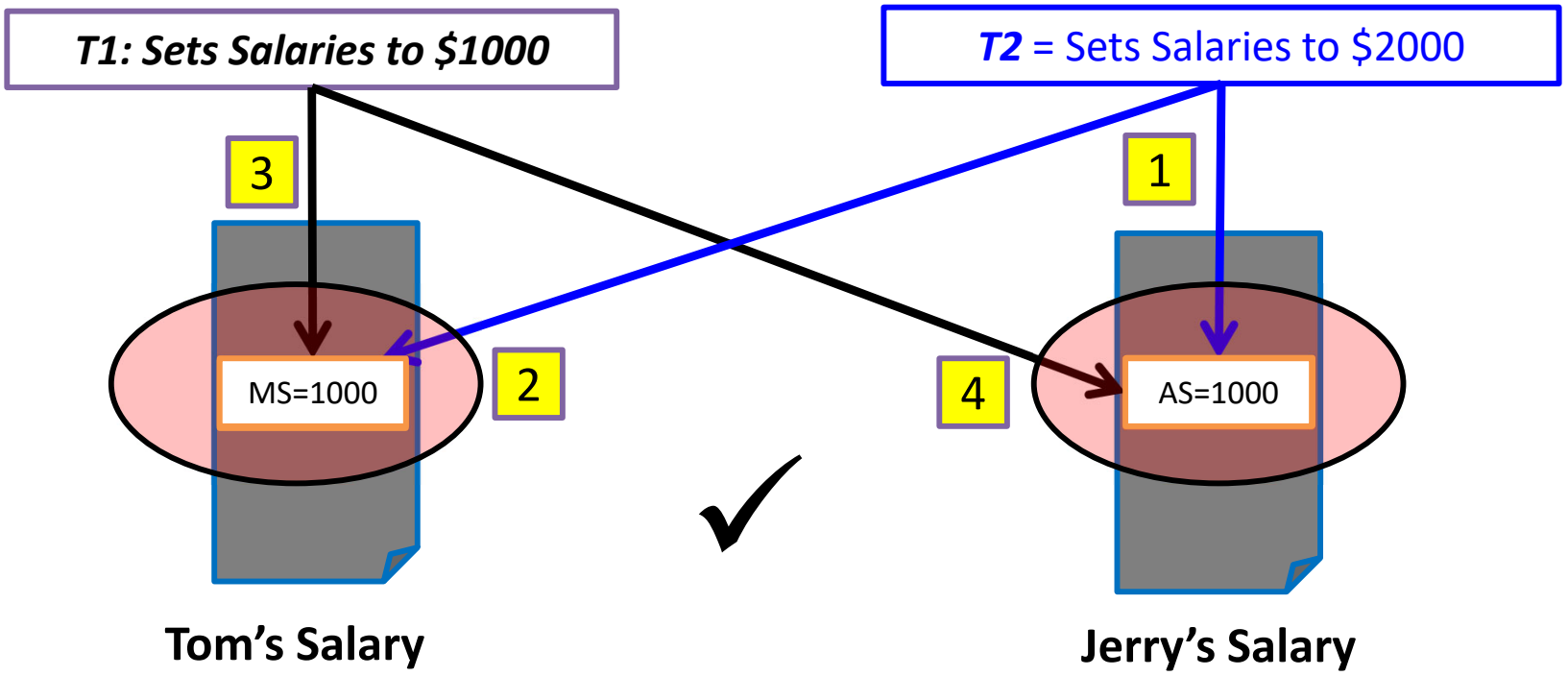


Jerry's Salary

Overwriting Uncommitted Data: WW Conflicts



Overwriting Uncommitted Data: WW Conflicts



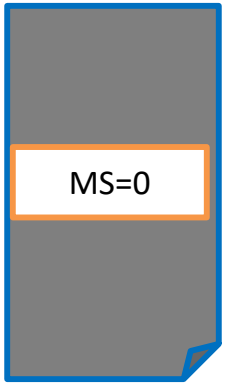
Either serial schedule is acceptable from a *consistency standpoint* (although Tom and Jerry may prefer higher salaries!)

Neither T1 nor T2 reads a salary value before writing it- such a write is called a ***blind write*** *盲写!*

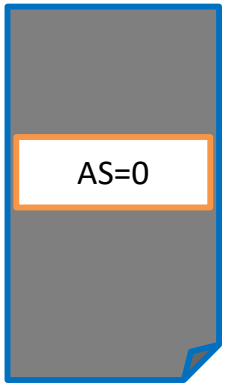
Overwriting Uncommitted Data: WW Conflicts

T1: Sets Salaries to $1000

T2 = Sets Salaries to $2000

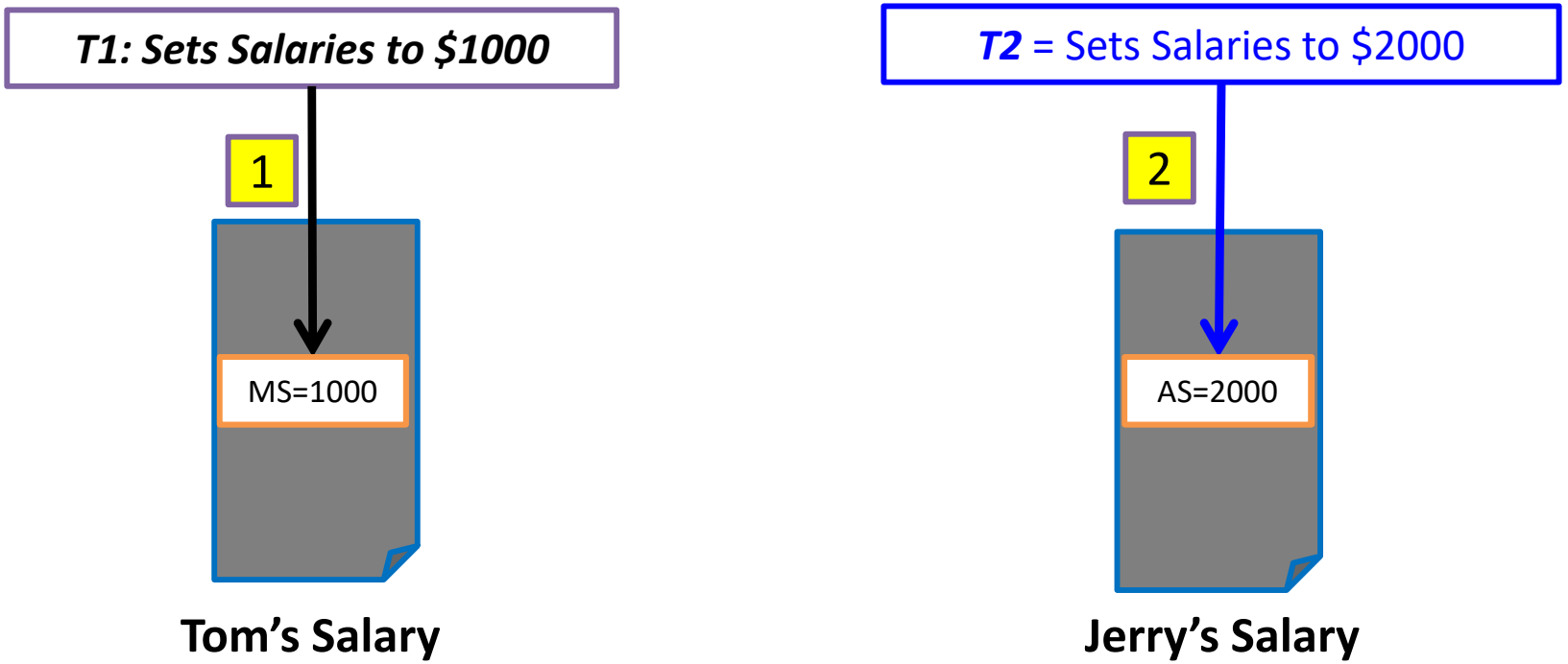


Tom's Salary

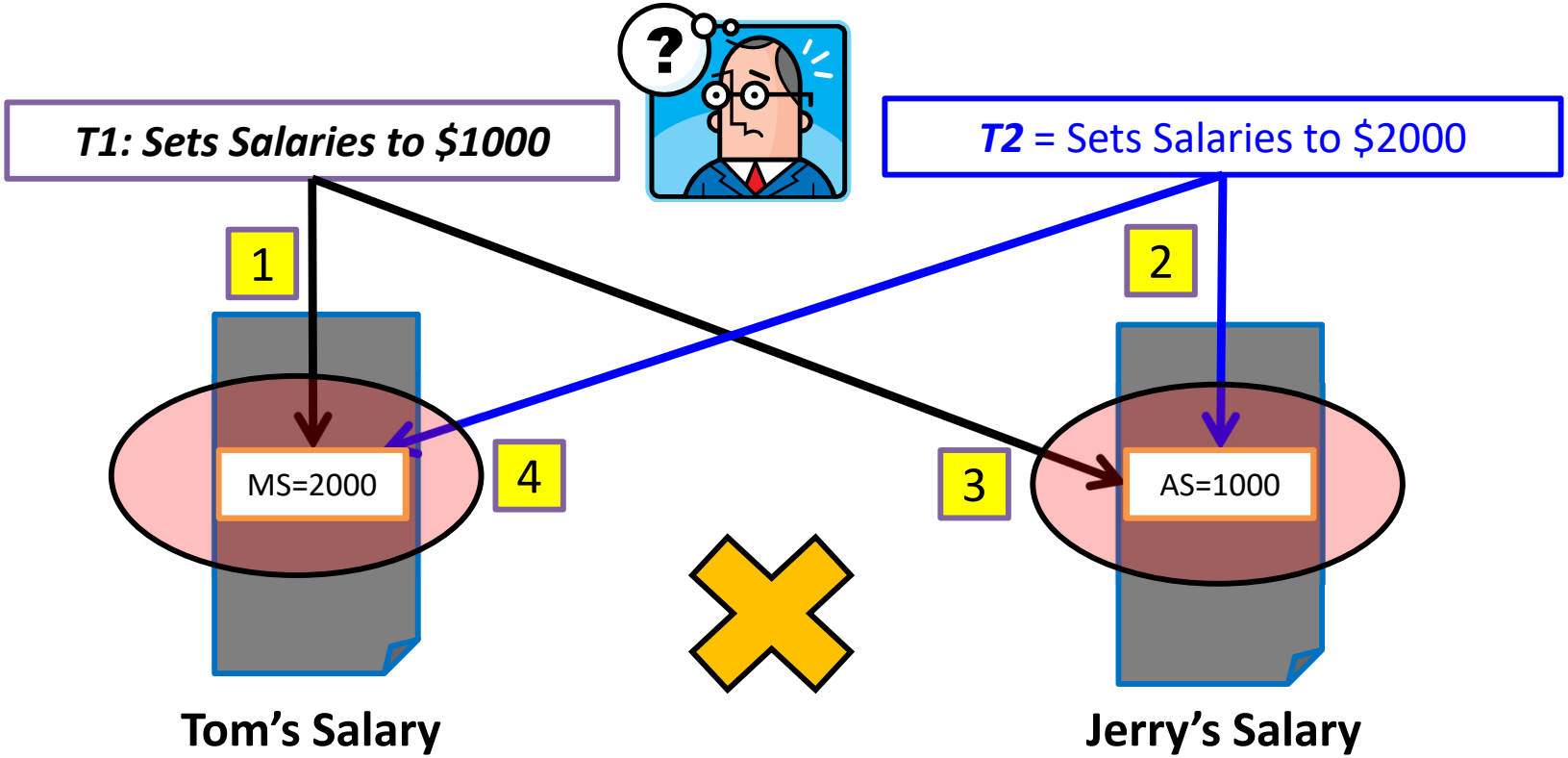


Jerry's Salary

Overwriting Uncommitted Data: WW Conflicts

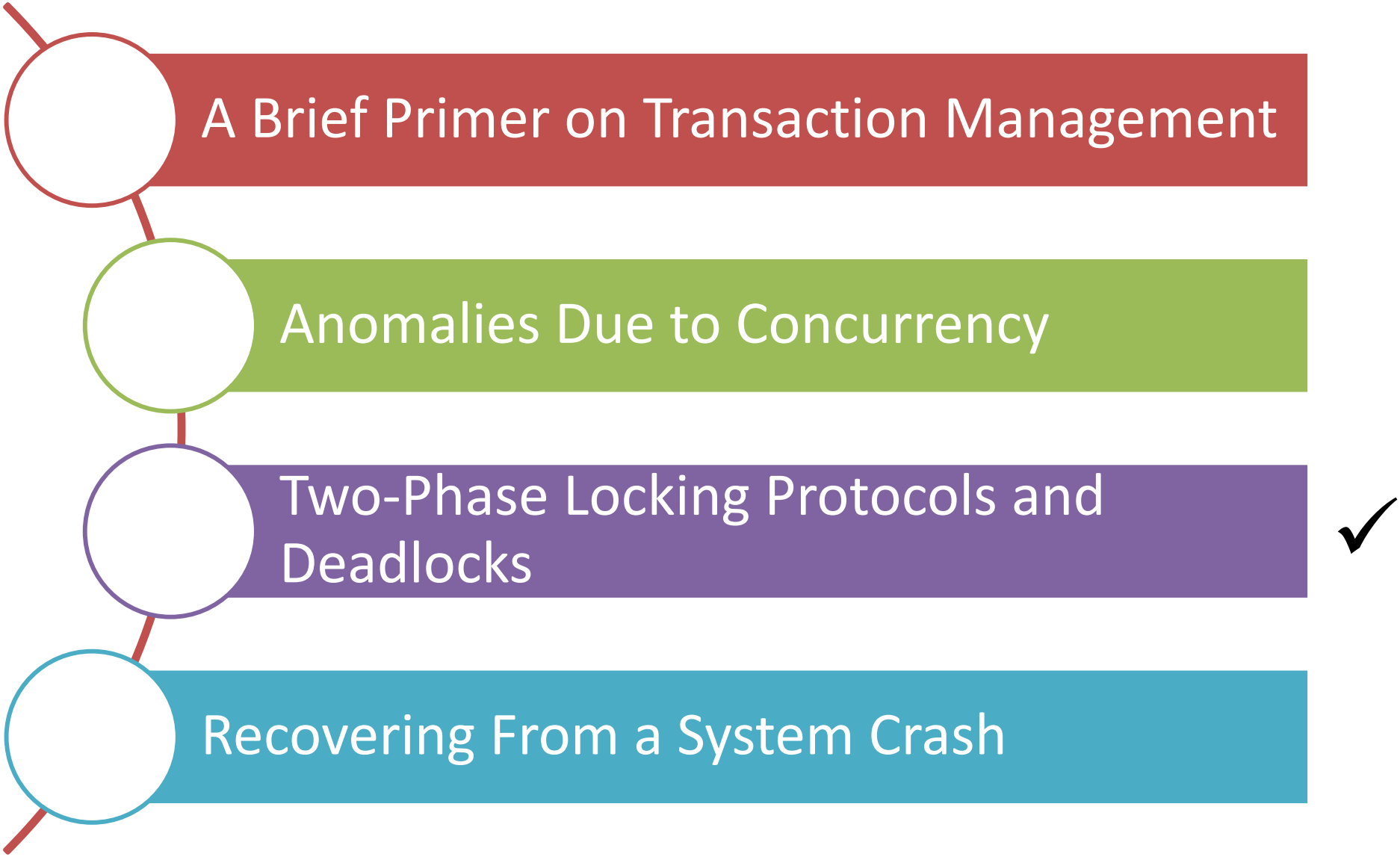


Overwriting Uncommitted Data: WW Conflicts



The problem is that we have a ***lost update*** 丢失的更新.
In particular, T2 overwrote Tom's Salary as set by T1
(this will never happen with a serializable schedule!)

Outline


A Brief Primer on Transaction Management
Anomalies Due to Concurrency
Two-Phase Locking Protocols and Deadlocks
Recovering From a System Crash

Locking Protocols

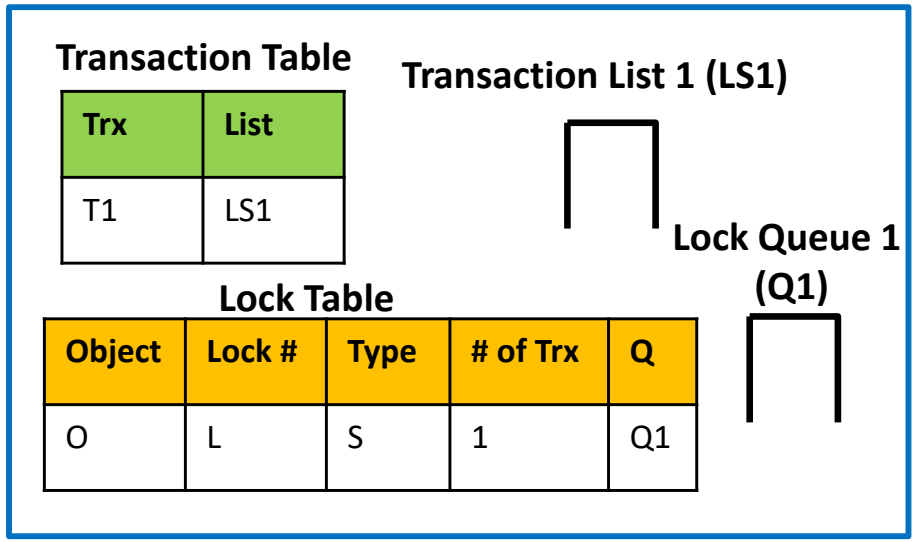
- WR, RW and WW anomalies can be avoided using a *locking protocol* 加锁协议

- A locking protocol:
  - Is *a set of rules* to be followed by each transaction to ensure that only serializable schedules are allowed
  - Associates a *lock* with each database object, which could be of different types (e.g., *shared* 共享 or *exclusive* *排他*)
  - *Grants and denies locks* to transactions according to the specified rules

- The part of the DBMS that keeps track of locks is called the *lock manager*

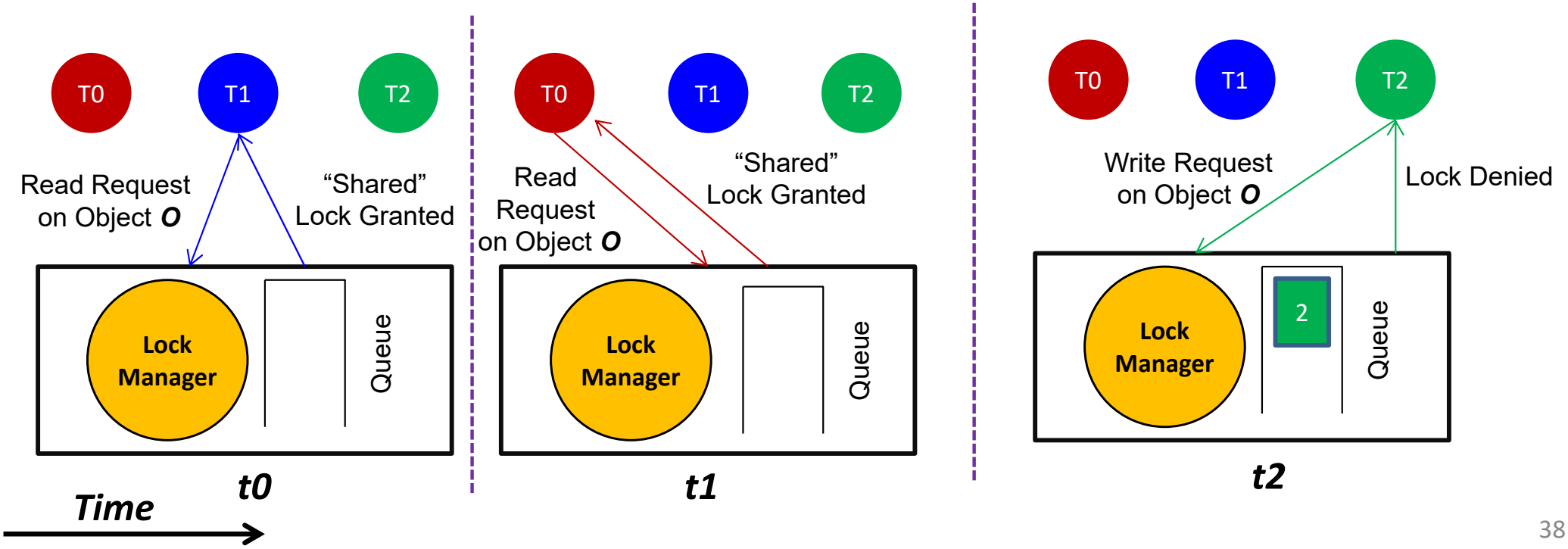
Lock Managers

- Usually, a lock manager in a DBMS maintains three types of data structures:
  - A queue, ***Q***, for each lock, ***L***, to hold its pending requests
  - A lock table, which keeps for each ***L*** associated with each object, ***O***, a record ***R*** that contains:
    - The type of ***L*** (e.g., shared or exclusive)
    - The number of transactions currently holding ***L*** on ***O***
    - A pointer to ***Q***
  - A transaction table, which maintains for each transaction, ***T***, a pointer to a list of locks held by ***T***



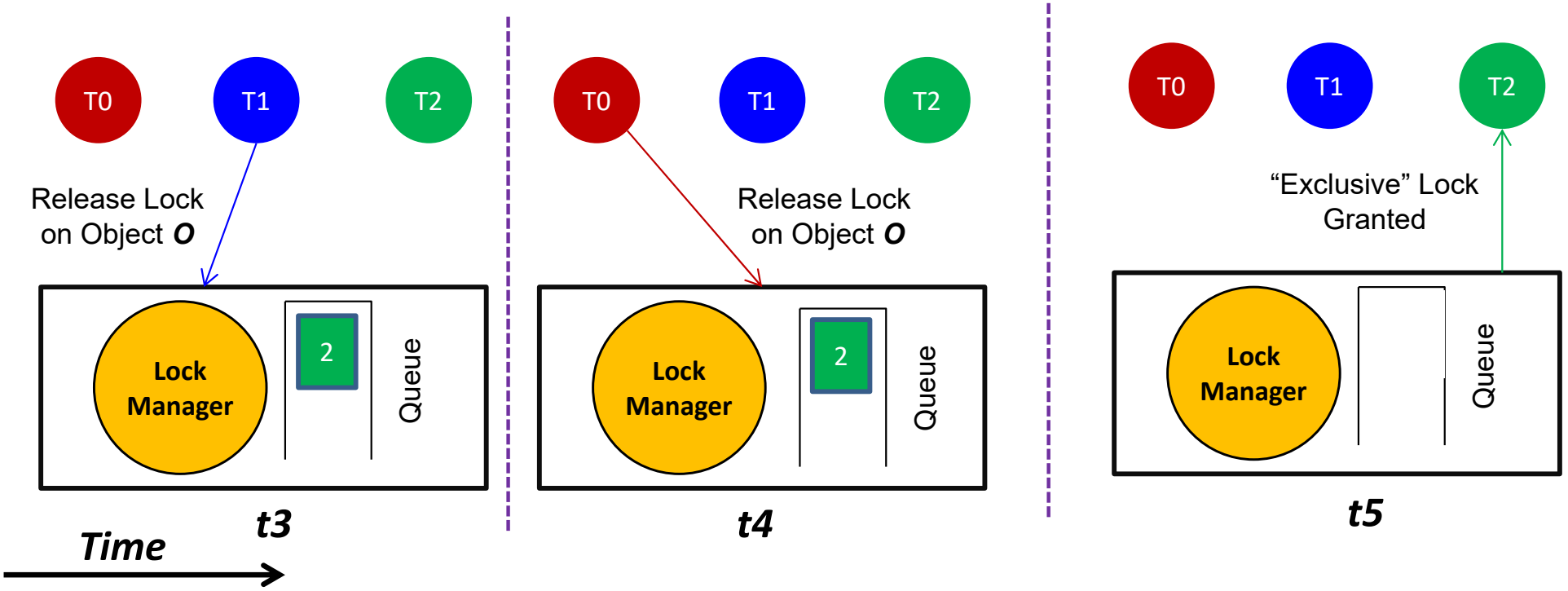
Two-Phase Locking

- A widely used locking protocol, called *Two-Phase Locking* (*2PL*), has two rules:
  - Rule 1: if a transaction ***T*** wants to read (or write) an object ***O***, it first requests the lock manager for a shared (or exclusive) lock on ***O***



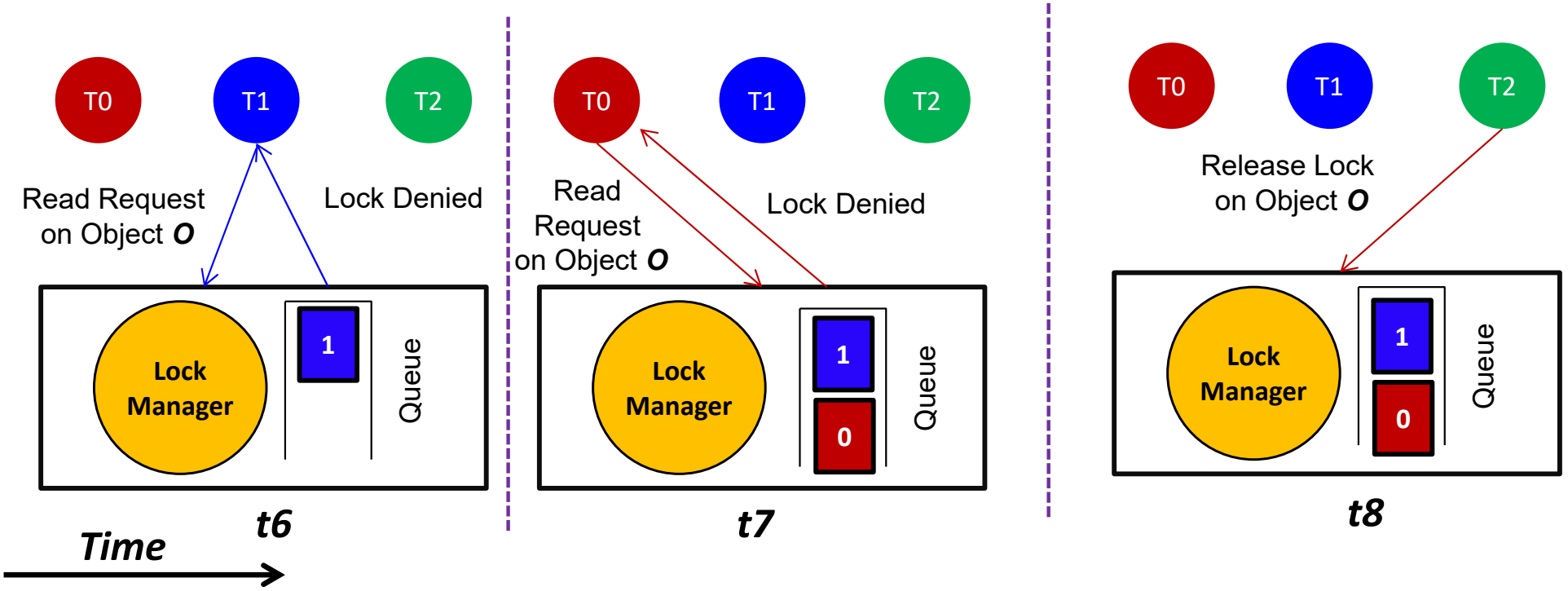
Two-Phase Locking

- A widely used locking protocol, called *Two-Phase Locking* (*2PL*), has two rules:
  - Rule 1: if a transaction ***T*** wants to read (or write) an object ***O***, it first requests the lock manager for a shared (or exclusive) lock on ***O***



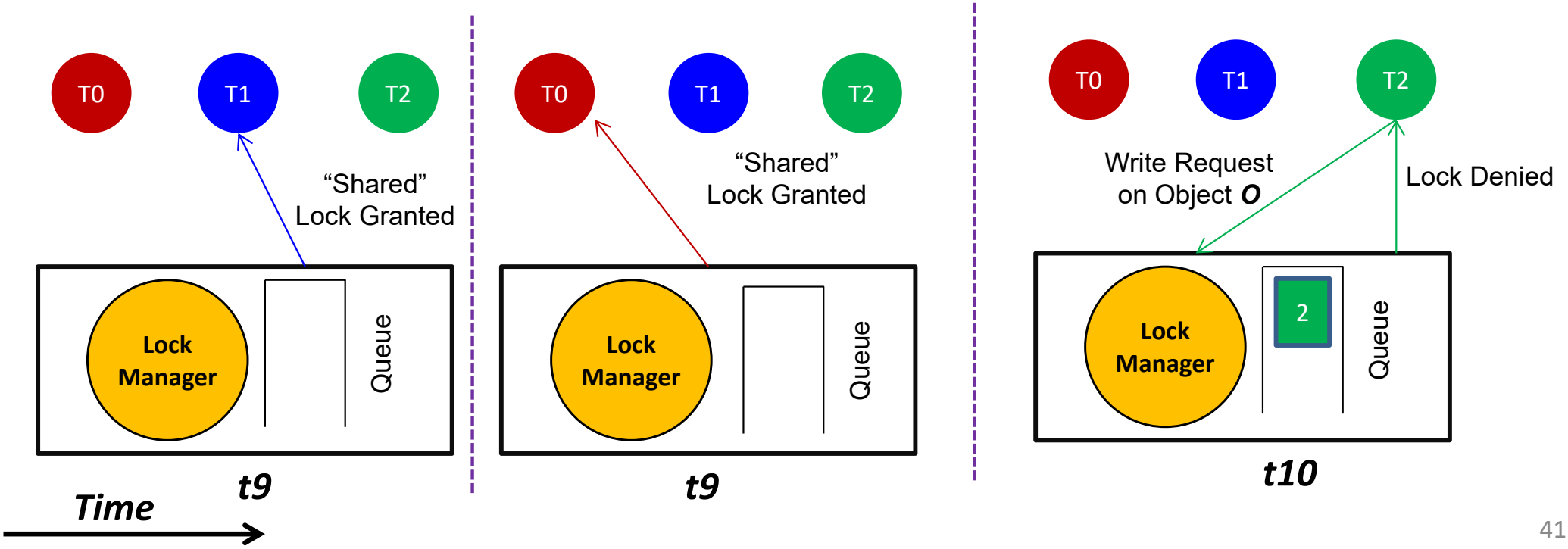
Two-Phase Locking

- A widely used locking protocol, called *Two-Phase Locking* (*2PL*), has two rules:
  - Rule 1: if a transaction ***T*** wants to read (or write) an object ***O***, it first requests the lock manager for a shared (or exclusive) lock on ***O***



Two-Phase Locking

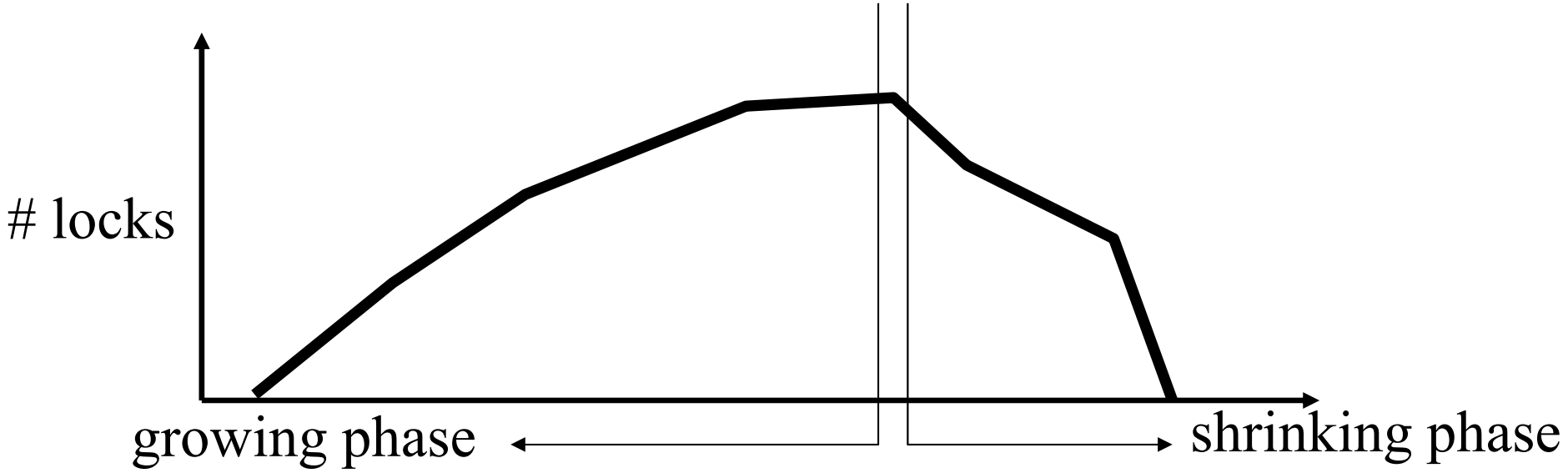
- A widely used locking protocol, called *Two-Phase Locking* (*2PL*), has two rules:
  - Rule 1: if a transaction ***T*** wants to read (or write) an object ***O***, it first requests the lock manager for a shared (or exclusive) lock on ***O***



Two-Phase Locking

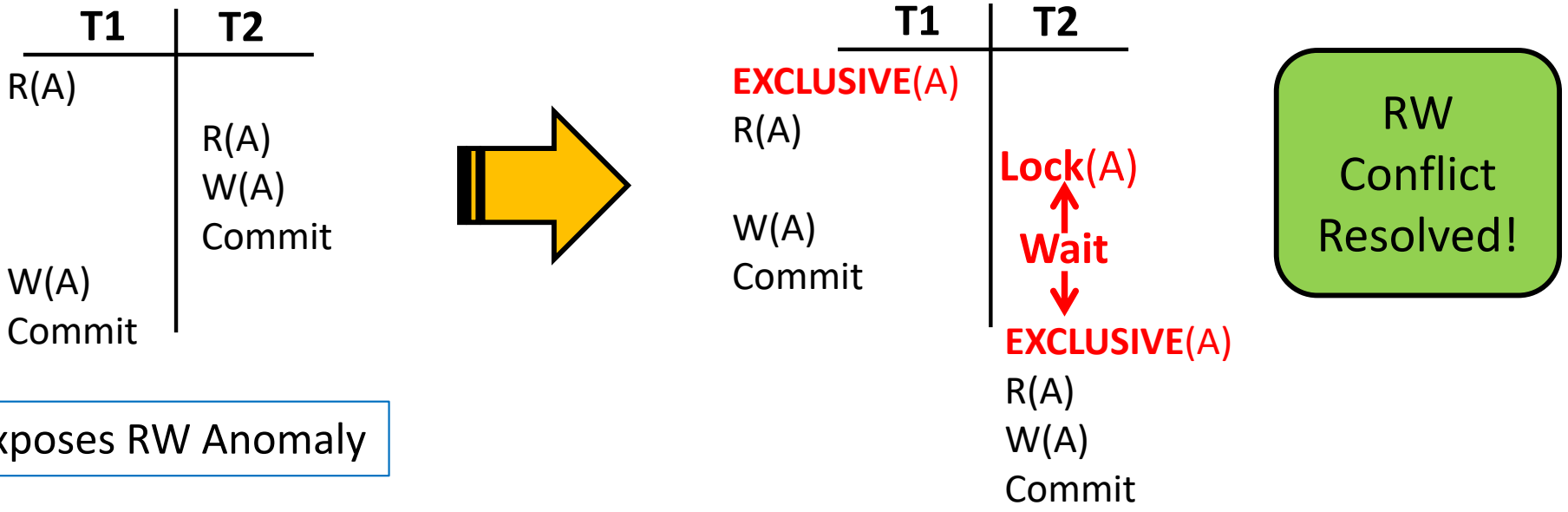
- A widely used locking protocol, called *Two-Phase Locking* (*2PL*), has two rules:
  - Rule 2: ***T*** can release locks before it *commits* or *aborts*, and cannot request additional locks once it releases *any* lock

- Thus, every transaction has a "growing" phase in which it acquires locks, followed by a "shrinking" phase in which it releases locks



Resolving RW Conflicts Using 2PL

- Suppose that T1 and T2 actions are interleaved as follows:
  - T1 reads A
  - T2 reads A, decrements A and commits
  - T1 tries to decrement A
- T1 and T2 can be represented by the following schedule:



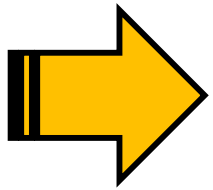
Resolving WR Conflicts Using 2PL

- Suppose that T1 and T2 actions are *interleaved* as follows:
  - T1 deducts $100 from account A
  - T2 adds 6% interest to accounts A and B
  - T1 credits $100 to account B

- T1 and T2 can be represented by the following schedule:

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
	R(B)
	W(B)
	Commit
R(B)	
W(B)	
Commit	

Exposes WR Anomaly



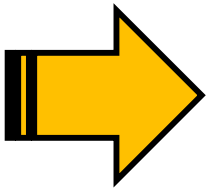
T1	T2
EXCLUSIVE(A)	
EXCLUSIVE(B)	**Lock**(A)
R(A)	**Lock**(B)
W(A)	↑
R(B)	**Wait**
W(B)	↓
Commit	**EXCLUSIVE**(A)
	EXCLUSIVE(B)
	R(A)
	W(A)
	R(B)
	W(B)
	Commit

WR Conflict Resolved!

Resolving WW Conflicts Using 2PL

- Suppose that T1 and T2 actions are interleaved as follows:
  - T1 sets Tom's Salary to $1000
  - T2 sets Jerry's Salary to $2000
  - T1 sets Jerry's Salary to $1000
  - T2 sets Tom's Salary to $2000
- T1 and T2 can be represented by the following schedule:

T1	T2
W(Tom)	
	W(Jerry)
W(Jerry)	
Commit	
	W(Tom)
	Commit



T1	T2
EXCLUSIVE(Tom)	
EXCLUSIVE(Jerry)	
W(Tom)	
W(Jerry)	
Commit	
	Lock(Jerry)
	Wait
	EXCLUSIVE(Jerry)
	EXCLUSIVE(Tom)
	W(Jerry)
	W(Tom)
	Commit

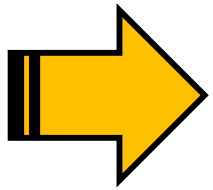
WW Conflict Resolved!

Exposes WW Anomaly
(*assuming, Tom & Jerry must be kept equal*)

Resolving WW Conflicts Using 2PL

- Suppose that T1 and T2 actions are interleaved as follows:
  - T1 sets Tom's Salary to $1000
  - T2 sets Jerry's Salary to $2000
  - T1 sets Jerry's Salary to $1000
  - T2 sets Tom's Salary to $2000
- T1 and T2 can be represented by the following schedule:

T1	T2
W(Tom)	
	W(Jerry)
W(Jerry)	
Commit	
	W(Tom)
	Commit



T1	T2
EXCLUSIVE(Tom)	
W(Tom)	
Lock(Jerry)	
↑	
Wait	
	EXCLUSIVE(Jerry)
	W(Jerry)
	Lock(Tom)
	↑
	Wait

Exposes WW Anomaly
(*assuming, Tom & Jerry must be kept equal*)

Deadlock!

Deadlock Detection

- The lock manager maintains a structure called a *waits-for graph* 等待图 to *periodically* detect deadlocks

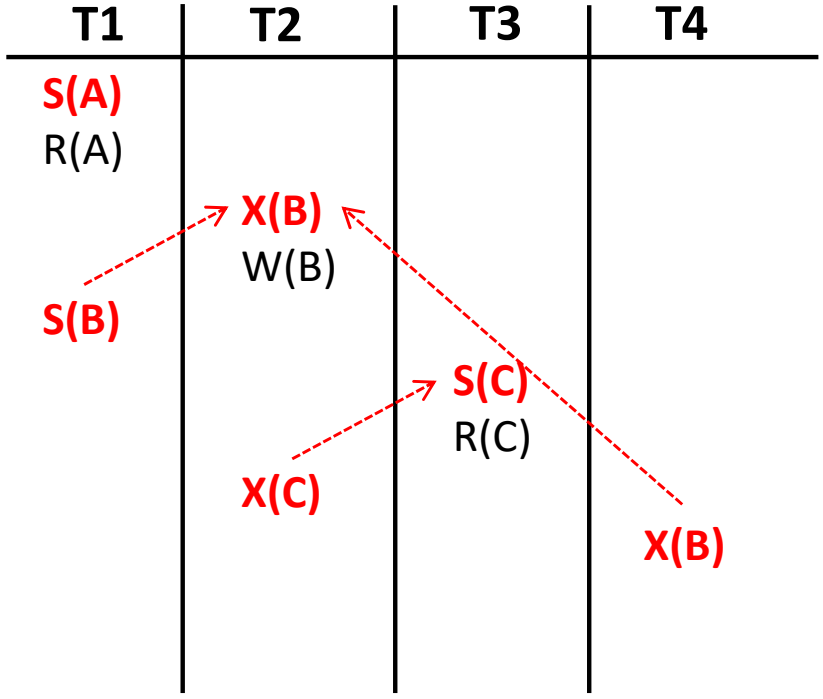
- In a waits-for graph:
  - The nodes correspond to active transactions
  - There is an edge from Ti to Tj *if and only if* Ti is waiting for Tj to release a lock

- The lock manager *adds* and *removes* edges to and from a waits-for graph when it *queues* and *grants* lock requests, respectively

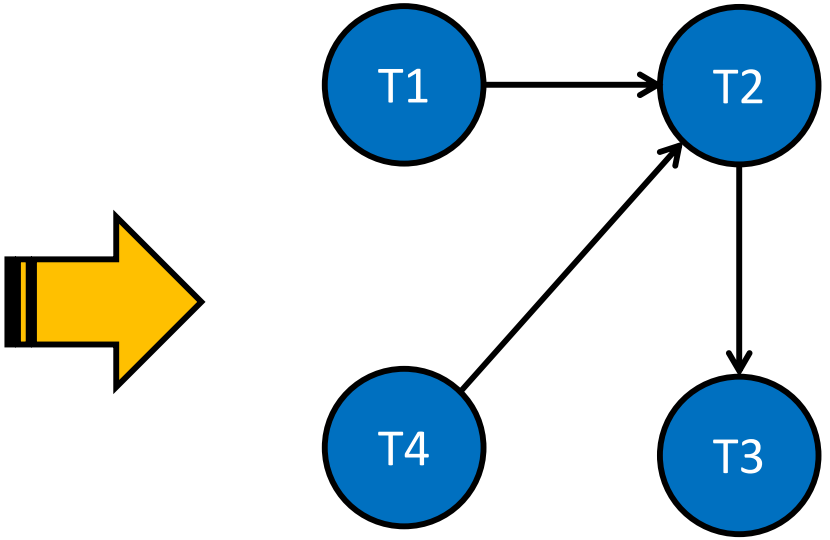
- A deadlock is detected when a *cycle* in the waits-for graph is found

Deadlock Detection (*Cont'd*)

- The following schedule is free of deadlocks:



A schedule *without* a deadlock



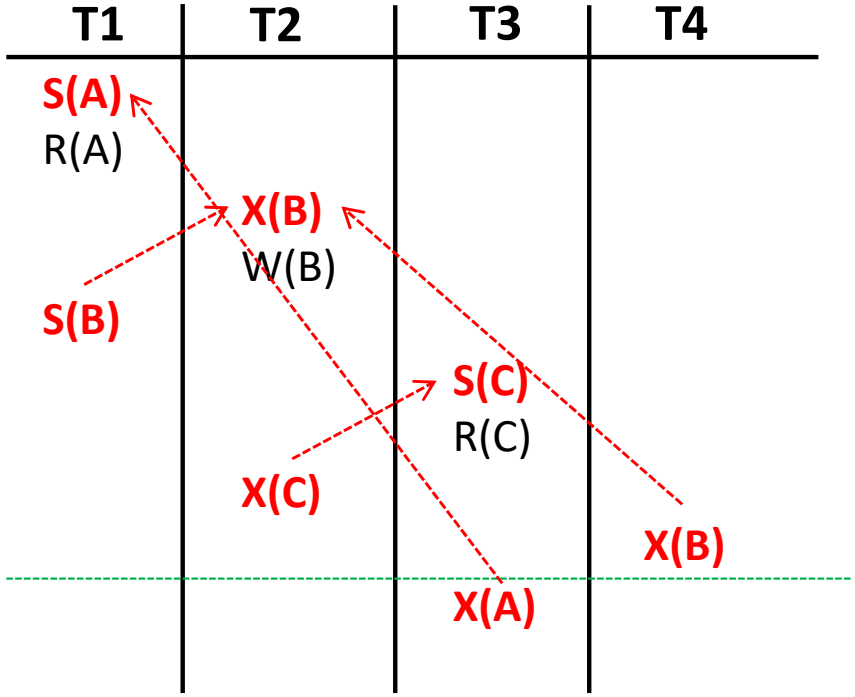
No cycles; hence, no deadlocks!

The Corresponding **Waits-For Graph***

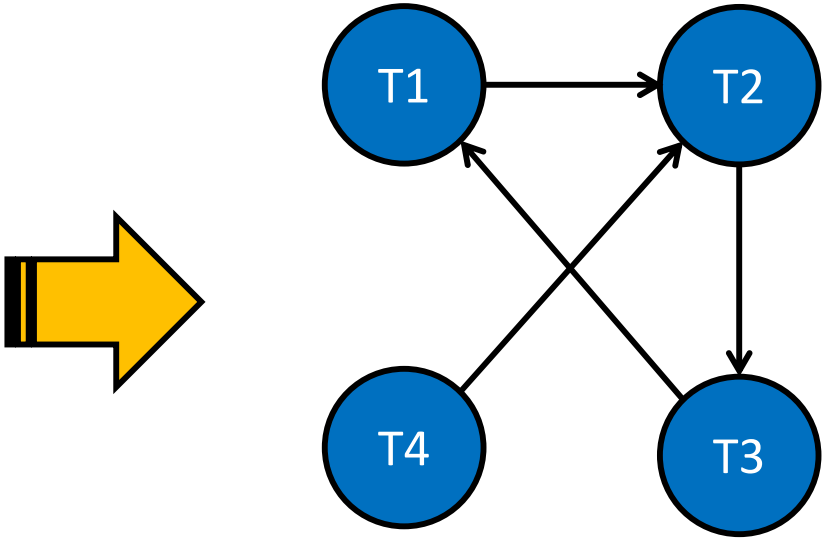
*The nodes correspond to active transactions and there is an edge from Ti to Tj *if and only if* Ti is waiting for Tj to release a lock

Deadlock Detection (*Cont'd*)

- The following schedule is **NOT** free of deadlocks:



A schedule *with* a deadlock

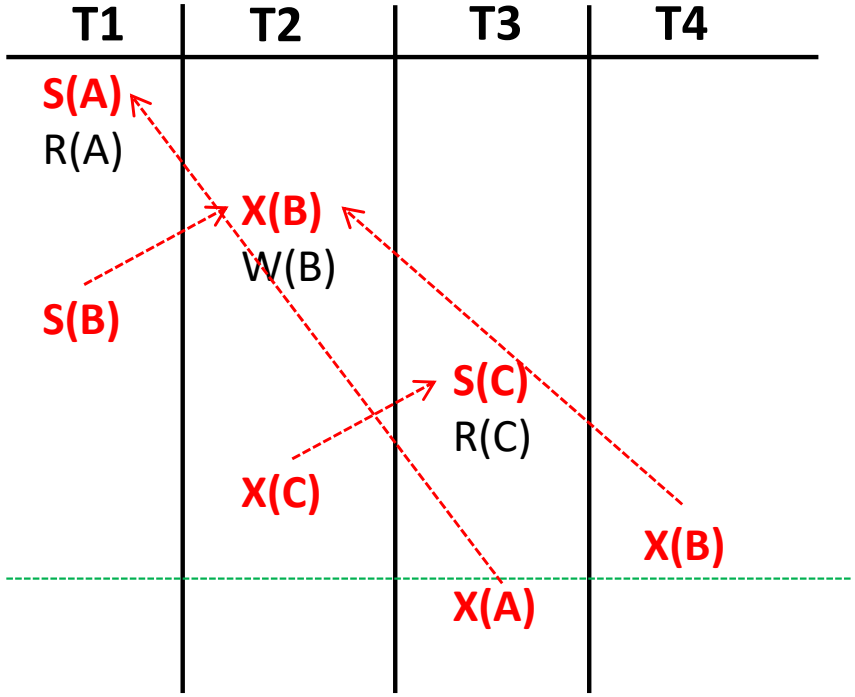


The Corresponding **Waits-For Graph***

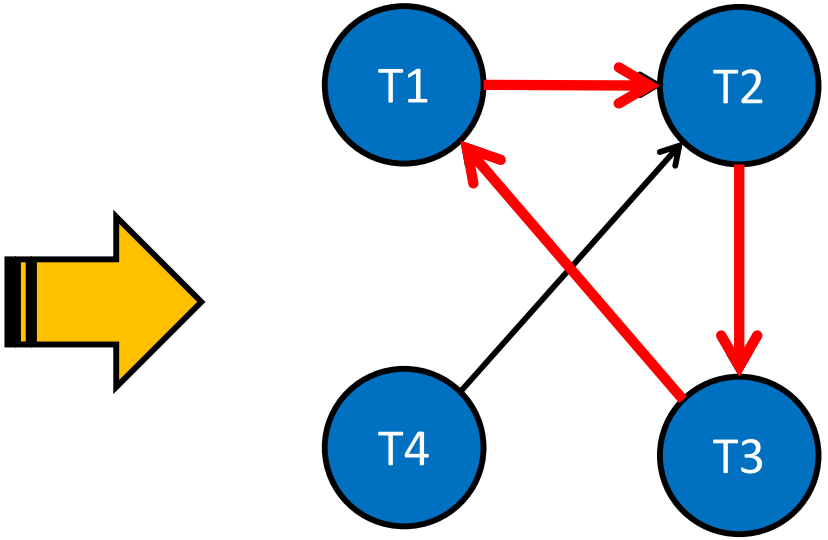
*The nodes correspond to active transactions and there is an edge from Ti to Tj *if and only if* Ti is waiting for Tj to release a lock

Deadlock Detection (*Cont'd*)

- The following schedule is **NOT** free of deadlocks:



A schedule *with* a deadlock



Cycle detected; hence, a deadlock!

The Corresponding **Waits-For Graph***

*The nodes correspond to active transactions and there is an edge from Ti to Tj *if and only if* Ti is waiting for Tj to release a lock

Resolving Deadlocks

- A deadlock is resolved by aborting a transaction that is on a cycle and releasing its locks
  - This allows some of the waiting transactions to proceed
- The choice of which transaction to abort can be made using different criteria:
  - The one with the fewest locks
  - Or the one that has done the least work
  - Or the one that is farthest from completion (*more accurate*)
- Alert: a transaction that was aborted in the past, should be *favored* subsequently and not aborted upon a deadlock detection!

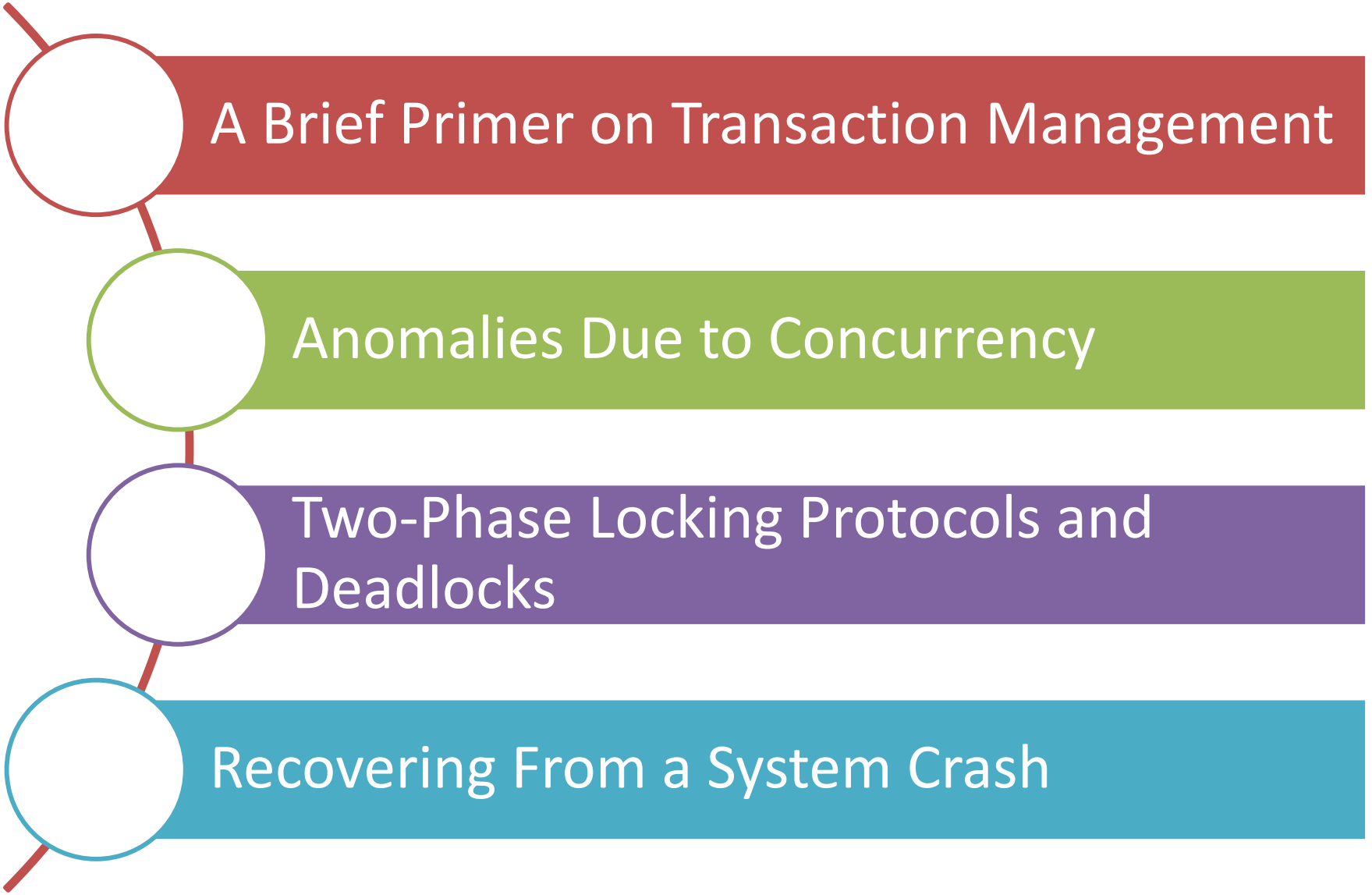
Deadlock Prevention

- Studies indicate that deadlocks are relatively infrequent and *detection-based schemes* work well in practice

- However, if there is a high level of *contention* for locks, *prevention-based schemes* could perform better

- Deadlocks can be averted by giving each transaction a *priority* and ensuring that lower-priority transactions are not allowed to wait for higher-priority ones
  (or vice versa)

Outline


A Brief Primer on Transaction Management
Anomalies Due to Concurrency
Two-Phase Locking Protocols and Deadlocks
Recovering From a System Crash

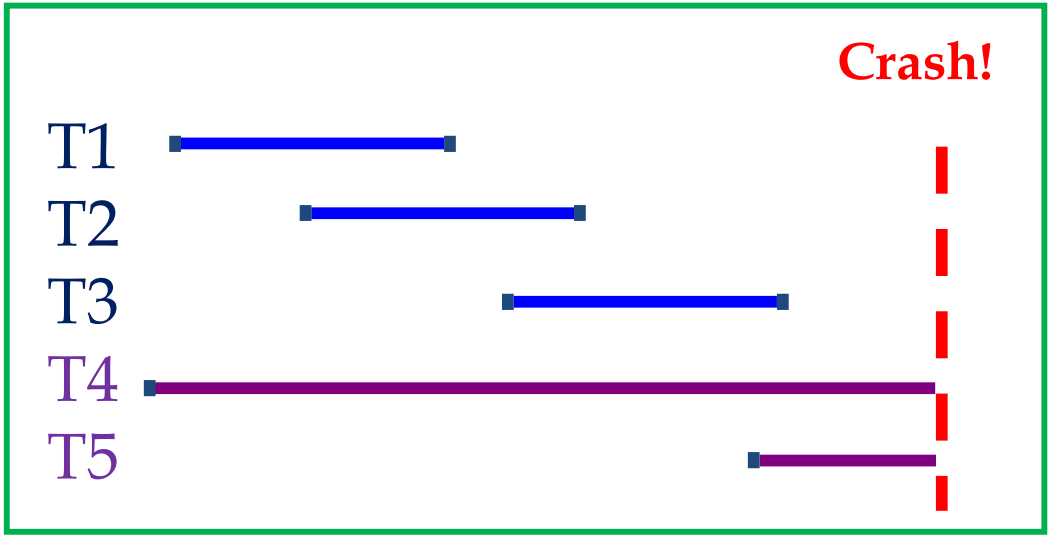
✓

The ACID Properties

- Four properties must be ensured in the face of concurrent accesses and system failures:
  - **A**tomicity原子性: Either all actions of a transaction are carried out or none at all
  - **C**onsistency一致性: Each transaction (run by itself with no concurrent execution) must preserve the consistency of the database
  - **I**solation隔离性: Execution of one transaction is isolated (or protected) from the effects of other concurrently running transactions
  - **D**urability持久性: If a transaction commits, its effects persist (even of the system crashes before all its changes are reflected on disk)

Ensuring Atomicity and Durability

- How can the recovery manager ensure atomicity and durability (in case of a failure)?
  - It can ensure atomicity by *undoing* the actions of transactions that did not commit
  - It can ensure durability by *redoing* (all) the actions of committed transactions



- Desired Behavior after the system restarts:
  - T1, T2 & T3 should be durable
  - T4 & T5 should be rolled back

Logging and the WAL Property

- In order to recover from failures, the recovery manager maintains a *log* of all modifications to the database on *stable storage* (which should survive crashes)

- After a failure, the DBMS "replays" the log to:
  - Redo committed transactions
  - Undo uncommitted transactions

- Alert: A log record describing a change must be written to stable storage *before* the change is made
  - This is referred to as the *Write-Ahead Log* (*WAL预写式日志*) *property*

The WAL Protocol

- WAL is the fundamental rule that ensures that a record of every change to the database is available after a crash
- What if a transaction made a change, committed, then a crash occurred (i.e., no log is kept "before" the crash)?
  - Without a record of this change, there would be no way to ensure that the committed transaction survives the crash
  - Hence, durability cannot be guaranteed!
- What if a transaction made a change, was progressing, and a crash occurred?
  - Without a record of this change, there would be no way to ensure that the transaction can be rolled back (i.e., its effects would be unseen)
  - Hence, atomicity cannot be guaranteed!

To guarantee ***durability & atomicity***, a record for every change must be written to stable storage *before the change is made*

Thank you!